

Parallel Time Integration of Dynamical Low Rank Approximation: Multirank Parareal

Benjamin Carrel

Master thesis under the supervision of
Prof. Martin J. Gander
Prof. Bart Vandereycken
(Université de Genève, Section de Mathématiques)

Summary

The dynamical low-rank approximation (DLRA) proposed in [9] is a new approach for computing efficiently large-scale problems. Many integrators have already been proposed, such as the projector splitting integrator [?], projected Runge–Kutta methods [8] and more recently an unconventional projector splitting integrator [2]. Since the fundamental problem is a time-dependent problem, it seems difficult to find a parallel integrator. Fortunately, for classical time-dependent differential equations, many parallel-time algorithms have been studied. Inspired by multigrid methods, we have a space-time parallel multigrid algorithm; see [6]. Inspired by domain-decomposition, we have the Schwarz waveform relaxation methods; see [4]. The Parareal algorithm [14] is probably one of the most popular time-parallel algorithms. In this thesis, we extend the idea of the Parareal algorithm to the dynamical low-rank approximation problem. We propose an analysis of the algorithm and numerical experiments on a parabolic problem and a parametric problem.

The thesis contains the following sections. Section 1 gives a historical overview of computing power. We give some ideas about efficient computing and we also define the notations used in the rest of the text. Section 2 explains the DLRA problem. After giving the ideas and the theory, we present three integrators and their properties. We also illustrate these integrators with numerical experiments. Section 3 is about the Parareal algorithm, its definition and its main properties. Section 4 describes a new algorithm for parallel time integration of DLRA. It is inspired by the Parareal algorithm, so we call it Multirank Parareal. We also give an analysis which leads to a first theoretical bound. Section 5 describes two problems on which we tested this new Multirank Parareal algorithm. The first problem is the Lyapunov equation applied to the heat problem. The second problem is a parametric problem for the heat diffusion. We also explain how to solve efficiently the substeps of the two projector splitting integrators given in Section 2. Section 6 is the conclusion.

Contents

1	Introduction	4
1.1	Efficient Computations	4
1.2	Notations	6
2	Dynamical Low Rank Approximation	8
2.1	The idea of DLRA	8
2.2	Theory and assumptions	9
2.3	Projector splitting integrator	12
2.3.1	Projector splitting: theory	13
2.3.2	Projector splitting: algorithm	14
2.3.3	KSL: Lyapunov	16
2.3.4	KSL: Cookie problem	18
2.4	Robust projector splitting integrator	19
2.4.1	KLS: Lyapunov	20
2.4.2	KLS: Cookie problem	20
2.5	Projected Runge–Kutta methods	22
2.5.1	PRKs: theory	22
2.5.2	PRKs: algorithm	25
2.5.3	PRKs: order of the methods	25
2.5.4	PRKs: Lyapunov	25
2.5.5	PRKs: Cookie problem	27
3	The Parareal algorithm	28
3.1	Parareal: definition	29
3.2	Parareal: properties	30
4	Parallel time integration of DLRA	35
4.1	A new interpretation of Parareal: Multirank Parareal . .	35
4.2	Analysis	37
5	Numerical experiments	49
5.1	Lyapunov Equation	49
5.1.1	Lyapunov: Heat problem	49
5.1.2	Lyapunov: computing the substeps	49
5.1.3	Lyapunov: Multirank Parareal	52
5.2	Cookie problem	53

5.2.1	Definition	53
5.2.2	Cookie problem: computing the substeps	59
5.2.3	Cookie problem: Multirank Parareal	62
6	Conclusion	65

1 Introduction

1.1 Efficient Computations

In the last decades, computers have become more and more powerful. Based on the website top500.org, the most powerful computer in June 2000 was the ASCI Red with 9 632 cores for a total of 2 379 GFlop/s. In June 2020, the most powerful computer is the Supercomputer Fugaku with 7 299 072 cores and 415 530 TFlop/s. In 20 years, the power of the best computer has been multiplied by a factor 175 000. To profit from all these cores, we need efficient algorithms that can run in parallel. Since we are running more and more precise simulations and are dealing with more and more data, an efficient implementation is necessary. Efficient computing methods try to answer the following questions:

1. How can we perform all of these computations as fast as possible?
2. How can we find the most interesting part in the data?
3. More generally, how can we store big data sets efficiently?

Some techniques are very simple and can already make a difference. Here are a few examples.

- *Efficient multiplications.* For two matrices $A, B \in \mathbb{R}^{n \times n}$ and a vector $\mathbf{v} \in \mathbb{R}^n$, computing $(AB)\mathbf{v}$ is an $O(n^3 + n^2)$ cost but computing $A(B\mathbf{v})$ is an $O(2n^2)$ cost, which makes a huge difference.
- *Factorization.* Let $A \in \mathbb{R}^{m \times n}$ be of rank r . Then, there exist two matrices $B \in \mathbb{R}^{m \times r}, C \in \mathbb{R}^{r \times n}$ such that $A = BC$. The memory needed for storing the matrix A in standard double precision is $m * n * 8$ bytes. Storing $A = BC$ in factorized form requires $m * r * 8 + r * n * 8$ bytes. If $r \ll m, n$; we can save a lot of memory.
- *Efficient additions.* The addition of two factorized forms $A = QR, B = \tilde{Q}\tilde{R}$ can be done without forming neither A nor B . We can just concatenate the factorizations in this way:

$$A + B = QR + \tilde{Q}\tilde{R} = \begin{bmatrix} Q & \tilde{Q} \end{bmatrix} \begin{bmatrix} R \\ \tilde{R} \end{bmatrix} =: \hat{Q}\hat{R}.$$

We can also do it with a factorization of the form $A = USV^T$, $B = \tilde{U}\tilde{S}\tilde{V}^T$. In this case, we have

$$A + B = USV + \tilde{U}\tilde{S}\tilde{V}^T = \begin{bmatrix} U & \tilde{U} \end{bmatrix} \begin{bmatrix} S & 0 \\ 0 & \tilde{S} \end{bmatrix} \begin{bmatrix} V & \tilde{V} \end{bmatrix}^T = \hat{U}\hat{S}\hat{V}^T.$$

Other techniques are less obvious. For example, how can we find the most important part of a given matrix $A \in \mathbb{R}^{m \times n}$? To answer this question, we need the two following theorems.

Theorem 1.1 (Singular value decomposition) *Let $A \in \mathbb{R}^{m \times n}$, with $m \geq n$. Then,*

$$A = U\Sigma V^T, \Sigma = \begin{bmatrix} \Sigma_1 \\ 0 \end{bmatrix} \in \mathbb{R}^{m \times n}, U \in \mathbb{R}^{m \times m}, V \in \mathbb{R}^{n \times n},$$

where $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ are orthonormal matrices. Moreover, $\Sigma_1 = \text{diag}(\sigma_1(A), \dots, \sigma_n(A))$, and $\sigma_i(A) = \lambda_i(AA^T)$ is called the i th singular value of A . This decomposition of A is known as the singular value decomposition (SVD) of A .

It is possible to truncate the SVD of A to the rank r . The truncated SVD is denoted by $A_{[r]}$, and

$$A_{[r]} = U_r \Sigma_r V_r^T, \text{ where } A = \begin{bmatrix} U_r & \tilde{U} \end{bmatrix} \begin{bmatrix} \Sigma_r & 0 \\ 0 & \tilde{\Sigma} \end{bmatrix} \begin{bmatrix} V_r^T \\ \tilde{V}^T \end{bmatrix}.$$

The following theorem shows that the most important part of a matrix A is given in its SVD. It is also called the Eckart–Young–Mirsky Theorem, see [3].

Theorem 1.2 (Truncated SVD as best rank r approximation) *Let $A \in \mathbb{R}^{m \times n}$ and $A_{[r]}$ be its rank- r truncated SVD. Then,*

$$\min_{X \in \mathbb{R}^{m \times n}} \{ \|A - X\| \mid \text{rank}(X) \leq r \} = \|A - A_{[r]}\|,$$

for any unitarily invariant norm $\|\cdot\|$ (like the spectral norm and the Frobenius norm).

Remark 1.1 *If A is a tensor, we only have a result of quasi-optimality.*

Having $A \approx U_k S_k V_k^T$ allows us to store efficiently $A \in \mathbb{R}^{m \times n}$ and perform additions and multiplications in an efficient way as explained above. Moreover, we keep a control on the error because we know that $A = U_k S_k V_k^T + \mathcal{E}$ where $\|\mathcal{E}\|_2 = \sigma_{k+1}(A) = \lambda_{k+1}(AA^T)$.

The main idea to keep in mind is that, in practice, one should never assemble big matrices and always keep and work with their factorized form.

1.2 Notations

In the rest of the text, we will use the following notations.

- The i th singular value of A is denoted by $\sigma_i(A) = \lambda_i(AA^T)$.
- The Frobenius norm is denoted by $\|\cdot\|_F$, i.e., $\|A\|_F^2 = \text{Tr}(AA^T)$. Important result: if $A \in \mathbb{R}^{m \times n}$, $m > n$ and $A = U\Sigma V^T$ is its SVD, then $\|A\|_F^2 = \|U\Sigma V^T\|_F^2 = \sum_{i=1}^n \sigma_i^2(A)$.
- The spectral norm is denoted by $\|\cdot\|_2$, i.e., $\|A\|_2^2 = \max_{i \in \{1, \dots, n\}} \{\lambda_i(AA^T)\}$. Important result: if $A \in \mathbb{R}^{m \times n}$, $m > n$ and $A = U\Sigma V^T$ is its SVD, then $\|A\|_2^2 = \|U\Sigma V^T\|_2^2 = \max_{i \in \{1, \dots, n\}} \{\sigma_i^2(A)\}$.
- When the choice of the norm is not of main concern, we simply use $\|\cdot\|$ to represent any unitarily invariant norm.
- If $Q \in \mathbb{R}^{m \times r}$ is an orthonormal matrix, the projection of $A \in \mathbb{R}^{m \times n}$ onto the column space of Q is denoted by $P_Q(A) = QQ^T A$.
- The manifold of rank- r matrices is defined by

$$\mathcal{M}_r = \{A \in \mathbb{R}^{m \times n} \mid \text{rank}(A) = r\}.$$

- The truncation (or retraction) to the manifold of rank- r matrices $\mathcal{M}_r$ is denoted by

$$\mathcal{T}_r : \mathbb{R}^{m \times n} \rightarrow \mathcal{M}_r.$$

By Theorem 1.2, the truncation of A is given by its truncated SVD. An other practical way to compute the truncation is to use the projection on the first r singular vectors of A .

- The tangent space to the manifold of rank- r matrices, in the matrix $Y \in \mathcal{M}_r$, is defined by

$$\mathcal{T}_Y \mathcal{M}_r = \{\dot{\gamma}(0) \mid \text{smooth curve } \gamma : \mathbb{R} \rightarrow \mathcal{M}_r \text{ such that } \gamma(0) = Y\}.$$

- The projection onto the tangent space $\mathcal{T}_Y\mathcal{M}_r$ is

$$\mathcal{P}(Y)[\cdot] : \mathbb{R}^{m \times n} \rightarrow \mathcal{T}_Y\mathcal{M}_r.$$

For a matrix $Z \in \mathbb{R}^{m \times n}$, it can be computed by

$$\mathcal{P}(Y)[Z] = ZP_{Im(Y^T)} - P_{Im(Y)}ZP_{Im(Y^T)} + P_{Im(Y)}Z;$$

see [9] for more details.

2 Dynamical Low Rank Approximation

In this section, we give the ideas and the theory about the Dynamical Low Rank Approximation (DLRA). The original theory about DLRA comes from [9], to which we refer for more details. In Section 2.3, we describe the original projector splitting integrator, derived in [?]. In Section 2.4, we describe a robust projector splitting integrator, derived in [2]. In Section 2.5, we describe the projected Runge–Kutta methods, derived in [8]. After presenting the main ideas behind the integrators, we give their algorithms and propose numerical experiments. These numerical experiments will also be useful in Section 5.

2.1 The idea of DLRA

As introduced in Section 1.1, an important idea of efficient computing is to use the low rank structures in the data. The DLRA follows this idea for time-dependent problems. In the literature (and more especially in physics), the DLRA is also called the Time Dependent Variational Principle (TDVP).

The fundamental problem to solve is either

- 1) a time dependent data $A(t)$ stored in a big matrix/tensor; or
- 2) a time dependent differential equation $\dot{A}(t) = F(t, A(t))$ involving big matrices/tensors.

Since

$$\dot{A}(t) \approx \frac{A(t+h) - A(t)}{h},$$

we will focus on the second problem. Wlog, from now on the problem to solve is the autonomous ODE

$$\begin{cases} \dot{A}(t) = F(A(t)) \text{ where } t \in [0, T], \\ A(0) = A_0. \end{cases} \quad (2.1)$$

Note that we will always consider the matrix case $A \in \mathbb{R}^{m \times n}$ for simplicity. However, most of the following results can be extended to tensors.

The goal is to find a good low-rank approximation, i.e.,

$$A(t) \approx X(t) \in \mathcal{M}_r, \text{ for some } r.$$

In other words, we are looking for

$$X(t) \in \mathcal{M}_r \text{ such that } \|X(t) - A(t)\| \stackrel{!}{=} \min. \quad (2.2)$$

By Theorem (1.2), we know that the optimal $X(t)$ is given by the truncated SVD of $A(t)$ for all $t \in [0, T]$. For tensors, we only have quasi-optimality.

Unfortunately, $A(t) \in \mathbb{R}^{m \times n}$ is a big matrix so computing its SVD for each t is computationally prohibitive. Even worse, since $A(t)$ is the solution of an ODE, we don't know its solution so we cannot apply faster SVD methods either, like HMT [7], Nyström [17] and SGN [16].

The dynamical low rank approximation solves both problems at the same time. The idea, proposed by Koch and Lubich in [9], is to find $Y(t) \in \mathcal{M}_r$ defined by

$$\dot{Y}(t) \in \mathcal{T}_{Y(t)}\mathcal{M}_r \text{ such that } \|\dot{Y}(t) - \dot{A}(t)\| = \|\dot{Y}(t) - F(A(t))\| \stackrel{!}{=} \min, \quad (2.3)$$

with an initial value, ideally $Y(0) = X(0) \approx A(0)$. Assuming that the solution of the original ODE (2.1) is unique on $[0, T]$, we prove in Theorem 2.2 that $A(t) \approx Y(t) \in \mathcal{M}_r$.

Define $\mathcal{P}(Y(t))[Z]$ the projection of Z onto the tangent space $\mathcal{T}_{Y(t)}\mathcal{M}_r$. Assuming that

$$A(t) \approx Y(t) \text{ so } \mathcal{P}(Y(t))[F(A(t))] \approx \mathcal{P}(Y(t))[F(Y(t))],$$

the DLRA problem 2.3 becomes the *DLRA ODE*

$$\begin{cases} \dot{Y}(t) = \mathcal{P}(Y(t))[F(Y(t))], \\ Y(0) = Y_0 \in \mathcal{M}_r, Y_0 \in \mathbb{R}^{m \times n}. \end{cases} \quad (2.4)$$

This way to formulate the problem is very elegant, and also very practical since we directly find a solution with low rank that is a good approximation of $A(t)$.

2.2 Theory and assumptions

Why does dynamical low rank approximation work? How much informations do we loose between solving (2.4) and solving the initial problem (2.2)? To answer these questions, we need to make the following assumptions on the ODE itself. The first two are standard, the third is

specific for DLRA. For simplicity, we sometimes omit the dependence on the time t , but the matrices can depend on t .

1. Wlog, the ODE is autonomous : $\dot{A} = F(A)$.
2. Lipschitz continuity of F , i.e., $\exists L, l \in \mathbb{R}, \forall X, Z \in \mathbb{R}^{m \times n}$,

$$\|F(X) - F(Z)\|_F \leq L \|X - Z\|_F, \quad (2.5)$$

$$\langle X - Z, F(X) - F(Z) \rangle_F \leq l \|X - Z\|_F^2. \quad (2.6)$$

3. F stays close to the tangent space of $\mathcal{M}_r$:

$$\|F(Z) - \mathcal{P}(Z)[F(Z)]\|_F \leq \varepsilon, \quad (2.7)$$

where ε is small for

$$Z \in \mathcal{M}_r \cap \{\text{suitable neighbourhood of the exact solution } A(t)\}.$$

Theorem 2.2 guarantees the accuracy of DLRA. Its proof requires the Grönwall lemma.

Lemma 2.1 (Grönwall lemma) *Let $u : [0, T] \rightarrow \mathbb{R}$ be a differentiable function such that*

$$\dot{u}(t) \leq a(t)u(t) + b(t),$$

where $a, b : [0, T] \rightarrow \mathbb{R}$ are continuous. Then, for all $t \in [0, T]$

$$u(t) \leq u(0) \exp \left(\int_0^t a(\kappa) d\kappa \right) + \int_0^t \exp \left(\int_s^t a(\kappa) d\kappa \right) b(s) ds.$$

Theorem 2.2 (Accuracy of DLRA) *Given the previous assumptions, and assuming that $A(t) \in \mathbb{R}^{m \times n}$ is the solution of the original ODE (2.1), then the solution $Y(t)$ of the DLRA ODE (2.4) satisfies*

$$\|A(t) - Y(t)\|_F \leq \|A(0) - Y(0)\|_F e^{lt} + \begin{cases} \varepsilon(e^{lt} - 1)l^{-1} & \text{if } l \neq 0 \\ \varepsilon t & \text{if } l = 0 \end{cases}.$$

Proof Let $E(t) = Y(t) - A(t)$ be the error. Then,

$$\frac{d}{dt} \|E(t)\|_F^2 = \frac{d}{dt} \langle E(t), E(t) \rangle = 2 \langle E(t), \dot{E}(t) \rangle. \quad (2.8)$$

By definition, we have

$$\begin{aligned}
\dot{E} &= \dot{Y} - \dot{A} \\
&= \mathcal{P}(Y) [F(Y)] - F(A) \\
&= \mathcal{P}(Y) [F(Y)] - F(Y) + F(Y) - F(A).
\end{aligned} \tag{2.9}$$

Then, inserting (2.9) into (2.8), we get

$$\begin{aligned}
\frac{d}{dt} \|E\|_F^2 &= \langle Y - A, \mathcal{P}(Y) [F(Y)] - F(Y) \rangle + \langle Y - A, F(Y) - F(A) \rangle \\
&\leq \|Y - A\|_F \|\mathcal{P}(Y) [F(Y)] - F(Y)\|_F + l \|Y - A\|_F^2 \\
&\leq \varepsilon \|E\|_F + l \|E\|_F^2.
\end{aligned}$$

Since

$$\frac{1}{2} \frac{d}{dt} \|E(t)\|_F^2 = \|E(t)\|_F \frac{d}{dt} \|E(t)\|_F,$$

we therefore obtain the differential inequality

$$\frac{d}{dt} \|E(t)\|_F \leq \varepsilon + l \|E\|_F,$$

which is solved by Grönwall's lemma 2.1 with $u(t) = \|E(t)\|_F$:

$$u(t) \leq u(0)e^{lt} + \varepsilon \int_0^t e^{l(t-s)} ds.$$

■

Since we know that the problem (2.4) makes sense to solve, we can give a first parametrization of the problem.

Theorem 2.3 (DLRA as three small and coupled ODEs) *Solving the DLRA ODE (2.4) given by*

$$\begin{cases} \dot{Y}(t) = \mathcal{P}(Y(t)) [F(Y(t))], \\ Y(0) = Y_0 \in \mathcal{M}_r, Y_0 \in \mathbb{R}^{m \times n}. \end{cases}$$

is equivalent to solving the three smaller coupled ODEs

$$\begin{cases} \dot{U}(t) = (I - P_{U(t)}) F_t V(t) S(t)^{-1}, \\ \dot{S}(t) = U(t)^T F_t V(t), \\ \dot{V}(t) = (I - P_{V(t)}) F_t^T U(t) S(t)^{-T}, \end{cases}$$

where

$$Y(t) = U(t)S(t)V(t)^T, \text{ and } F_t = F(U(t)S(t)V(t)^T).$$

The initial values of each ODE are given by

$$Y(0) = U(0)S(0)V(0)^T.$$

Proof Let $Y \in \mathbb{R}^{m \times n}$ and $Y \in \mathcal{M}_r$ be the solution of (2.4). By the SVD, we can write

$$Y = USV^T, \text{ where } U \in \mathbb{R}^{m \times r}, S \in \mathbb{R}^{r \times r}, V \in \mathbb{R}^{n \times r}.$$

Hence, taking the derivative and the definition of DLRA, we get

$$\mathcal{P}(Y)[F(Y)] = \dot{Y} = \dot{U}SV^T + U\dot{S}V^T + US\dot{V}^T.$$

The projection (see [9] for details) is given by

$$\mathcal{P}(Y)[Z] = P_U Z P_V + (I - P_U) Z P_V + P_U Z (I - P_V).$$

By the previous computation, we get

$$P_U F_t P_V + (I - P_U) F_t P_V + P_U F_t (I - P_V) = \dot{U}SV^T + U\dot{S}V^T + US\dot{V}^T.$$

The definition $P_U = UU^T$, $P_V = VV^T$ and the orthonormality of U and V lead to the system of equations

$$\begin{cases} \dot{U}SV^T = (I - P_U)F_t P_V, \\ U\dot{S}V^T = P_U F_t P_V, \\ US\dot{V}^T = P_U F_t (I - P_V). \end{cases}$$

Finally, using again the orthonormality of U and V , we get the three coupled ODEs given in the theorem. ■

2.3 Projector splitting integrator

In [?], Lubich and Oseledets proposed an integrator based on splitting the projectors. We will present key results, and give the algorithm, also called the KSL scheme. We finish this section with a numerical experiment.

2.3.1 Projector splitting: theory

From [?], we know that the projection onto the tangent space of $Y \in \mathcal{M}_r$ is

$$\mathcal{P}(Y)[Z] = ZP_{Im(Y^T)} - P_{Im(Y)}ZP_{Im(Y^T)} + P_{Im(Y)}Z.$$

Now, we can rewrite the DLRA ODE (2.4) using this formula,

$$\dot{Y} = \mathcal{P}(Y)[F(Y)] = F(Y)P_{Im(Y^T)} - P_{Im(Y)}F(Y)P_{Im(Y^T)} + P_{Im(Y)}F(Y).$$

We can use a Lie-Trotter scheme to solve this ODE, which leads to the following three small time-dependent ODEs

$$\begin{cases} \dot{Y}_I = F(Y_I)P_{Im(Y_I^T)}, \\ \dot{Y}_{II} = -P_{Im(Y_{II})}F(Y_{II})P_{Im(Y_{II}^T)}, \\ \dot{Y}_{III} = P_{Im(Y_{III})}F(Y_{III}), \end{cases} \quad (2.10)$$

with the initial value for each substep given by the Lie-Trotter scheme, that is,

$$\begin{cases} Y_I(0) = Y(0), \\ Y_{II}(0) = Y_I(h), \\ Y_{III}(0) = Y_{II}(h), \end{cases}$$

and finally,

$$Y(h) = Y_{III}(h).$$

Remark 2.1 *This is a scheme of order 1. Algorithm 2.1 uses this scheme. By symmetry it is easy to extend it to the Strang scheme which is of order 2, see [?, Section 3.3]. This is what we used in our experiments.*

The idea of Lubich and Osedeleks is to identify Y_I, Y_{II}, Y_{III} via the SVD of $Y(t)$. Let Y_0 be given by its SVD,

$$Y_0 = U_0 S_0 V_0^T, \text{ where } U_0 \in \mathbb{R}^{m \times r}, S_0 \in \mathbb{R}^{r \times r}, V_0 \in \mathbb{R}^{n \times r}.$$

In [?, Lemma 3.1], we set $Y_I = U_I S_I V_I^T$ and we assume that $\dot{V}_I = 0$ to obtain the solution of the first substep. This choice is efficient since we now have $V_I(t) = V_0$. So we simplified the previous expression to

$$Y_I(t) = U_I(t)S_I(t)V_0^T = K(t)V_0^T,$$

and therefore have $P_{Im(Y_I^T)} = V_0 V_0^T$, so by (2.10) we obtain the ODE

$$\dot{K}(t) = F(Y_I(t))V_0 I_r = F(K(t)V_0^T)V_0.$$

The same idea holds with $Y_{II}(t) = U_1 S(t)V_0^T$ and $Y_{III} = U_1 L(t)^T$ and leads to the three small ODEs

$$\begin{cases} \dot{K}(t) = F(K(t)V_0^T)V_0, & K(0) = U_0 S_0, & K(t) \in \mathbb{R}^{m \times r}, \\ \dot{S}(t) = -U_1^T F(U_1 S(t)V_0^T)V_0, & S(0) = R_1, & S(t) \in \mathbb{R}^{r \times r}, \\ \dot{L}(t) = F(U_1 L(t)^T)^T U_1, & L(0) = V_0 S(h)^T, & L(t) \in \mathbb{R}^{r \times n}. \end{cases} \quad (2.11)$$

The initial values are given using

$$\begin{aligned} K(t) = U(t)S(t) &\implies K(t_1) = U(t_1)S(t_1) = U_1 R_1 \text{ by QR,} \\ L(t) = V(t)S(t)^T &\implies L(t_1) = V(t_1)S(t_1)^T = V_1 S_1^T \text{ by QR.} \end{aligned}$$

Finally, we obtain

$$Y(h) = U_1 S_1 V_1^T.$$

2.3.2 Projector splitting: algorithm

Applying this process for many time steps is known as the projector splitting integrator, also called the KSL scheme. Its pseudocode is given in Algorithm 2.1.

Algorithm 2.1 The projector splitting integrator, or the KSL scheme.

Require: Initial value $Y_0 = U_0 S_0 V_0^T$, a time discretization $t_0 < t_1 < \dots < t_n$.

for $j=0,1,2,\dots,n-1$ **do**

Solve IVP : $\dot{K}(t) = F(K(t)V_j^T)V_j$ with $K(t_j) = U_j S_j$.

Compute QR : $K(t_{j+1}) = U_{j+1} R$.

Solve IVP : $\dot{S}(t) = -U_{j+1}^T F(U_{j+1} S(t)V_j^T)V_j$ with $S(t_j) = R$.

Solve IVP : $\dot{L}(t) = F(U_{j+1} L(t)^T)^T U_{j+1}$ with $L(t_j) = V_j S(t_{j+1})^T$.

Compute QR : $L(t_{j+1}) = V_{j+1} S_{j+1}^T$.

Result at step $(j+1)$: $Y_{j+1} = U_{j+1} S_{j+1} V_{j+1}^T$.

end for

Remark 2.2 • *The substep S is a backward step, so we have to be careful if the ODE is stiff, like the heat problem. This issue is addressed by the second integrator, given in Section 2.4.*

- *For convergence, each substep has to be solved with high precision. Computing these solutions can be expensive. In Section 5.1 and Section 5.2, we derived the closed-form solution of each substep for the two problems we considered. In our numerical experiments, we always used these closed-form solutions.*
- *The choice to solve K , S and L in order is important. It gives the smallest hidden constant in the analysis, and also in the numerical experiments.*

The two following results are theoretical results that insure a good behavior of the solution computed by Algorithm 2.1. Theorem 2.4 proves the exactness of Algorithm 2.1. Its proof is given in [?, Theorem 4.1].

Theorem 2.4 (Exactness property) *Suppose that $A(t)$ has rank at most r for all t . With the initial value $Y_0 = A(0)$, the splitting algorithm described above is exact, i.e., $Y_1 = A(h)$.*

Theorem 2.5 gives an error bound showing the robustness to small singular values of Algorithm 2.1. Its proof is given in [?, Theorem 4.2].

Theorem 2.5 (Error bound, robustness to small singular values) *Suppose $Y_0 \approx A(0)$ such that $\text{rank}(Y_0) = r$ and $\hat{Y}_0 \approx A(0)$ such that $\text{rank}(\hat{Y}_0) = q < r$. If Y_n and $\hat{Y}_n$ denote the results after n steps of the projector splitting integrator for the rank- r and rank- q approximations respectively, with a step size h , then, as long as $nh \leq T$,*

$$\left\| Y_n - \hat{Y}_n \right\|_F \leq C(\varepsilon + h), \quad (2.12)$$

where C is independent of n, h and ε (but depends on T). The parameter ε is a small perturbation, such that $A(t) = \tilde{A}(t) + \varepsilon \hat{A}(t)$ with $\text{rank}(\tilde{A}(t)) = q < r = \text{rank}(A(t)), \forall t \in [0, T]$.

In [18], Ostermann, Piazzola and Walach proposed an analysis of the KSL scheme. In particular, they obtained the following Theorem 2.6, which gives an error bound for the KSL scheme. As always, the roundoff error appears to be the minimal error possible.

Theorem 2.6 (Global error bound of the KSL scheme) *Let $A(t)$ be the exact solution of the problem (2.1). Let Y_n^h denote the results after*

n steps of the KSL scheme with time step h . Under the assumptions given in Section 2.2 and if the initial values verify

$$\|A_0 - Y_0\| \leq \delta,$$

there exists $h_0 > 0$ such that for all step sizes $0 < h < h_0$ the error of the KSL scheme is uniformly bounded on $0 \leq nh \leq T$ by

$$\|A(nh) - Y_n^h\| \leq c_0 h(1 + |\log(h)|) + c_1 \delta + c_2 \varepsilon,$$

where c_0, c_1 and c_2 are independent of h and n .

2.3.3 KSL: Lyapunov

In this section, we give a numerical experiment for Algorithm 2.1, modified for order 2. The substeps are solved with their closed-form solution (see Section 5.1). We used the Lyapunov equation applied to a heat problem, see Section 5.1 for details. For readability, we briefly state the problem here. The Lyapunov equation is

$$\dot{X}(t) = AX(t) + X(t)A + C,$$

for some matrices $A, C \in \mathbb{R}^{n \times n}$, and an initial value $X(0) = X_0$. If we set A to be the one-dimensional Laplacian, and C as the source, this equation solves the heat problem in two dimensions. The settings are :

- Size of the problem is $n = 50$.
- X_0 is a symmetric random matrix with exponentially decaying singular values. Its rank is $\text{rank}(X_0) = 10$.
- $C = \mathbf{u}\mathbf{u}^T$, for a random vector $\mathbf{u} \in \mathbb{R}^{50}$, so the source has rank 1.
- Initial time is $t_0 = 0$ and final time is $t_1 = 1$.
- For numerical reasons, we start using the solvers at time $t'_0 = 0.01$, with the exact initial value $X(t'_0)$.

Figure 2.1 represents the error of Algorithm 2.1 (the order 2 version) for this problem. As we can see, the algorithm has converged to the solution of the DLRA ODE (2.4). This solution is indeed quasi-optimal, since we are near the best rank- r approximation of the solution. The heat problem is a stiff problem, so we have to be careful because the KSL scheme contains a backward step. In Figure 2.2, with used only 40 time steps. Since the problem is stiff, the KSL scheme does not converge for the rank-8 approximation.

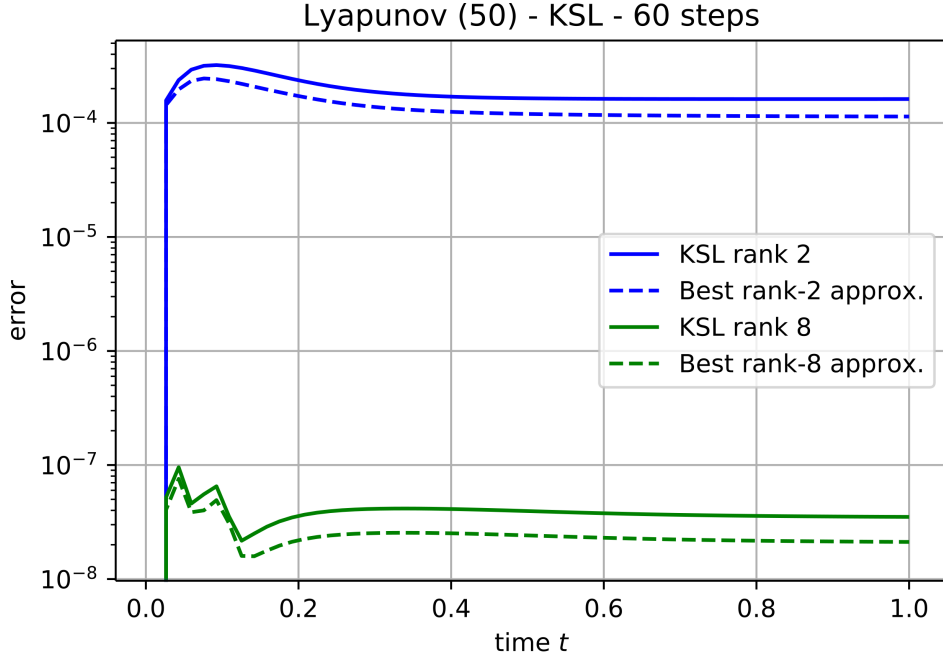


Figure 2.1: The KSL scheme of order 2, applied to the Lyapunov problem ($n=50$). The blue line is the error $\|X_2(t) - X(t)\|_F$. The blue dashed line is the error $\|Y_2(t) - X(t)\|_F$, where $Y_2(t)$ is the solution to the DLRA ODE with rank 2, computed by the KSL scheme. Idem for the green line with rank 8. With 60 time steps, the scheme has converged.

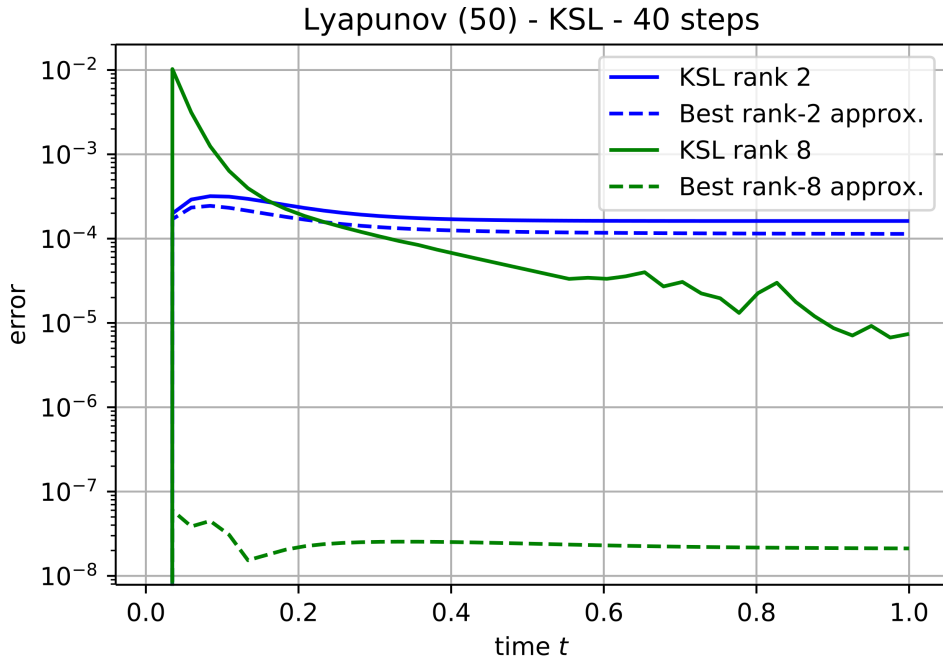


Figure 2.2: Illustration of the stiffness of the problem. With 40 time steps, the KSL scheme applied to the rank-8 approximation does not converge.

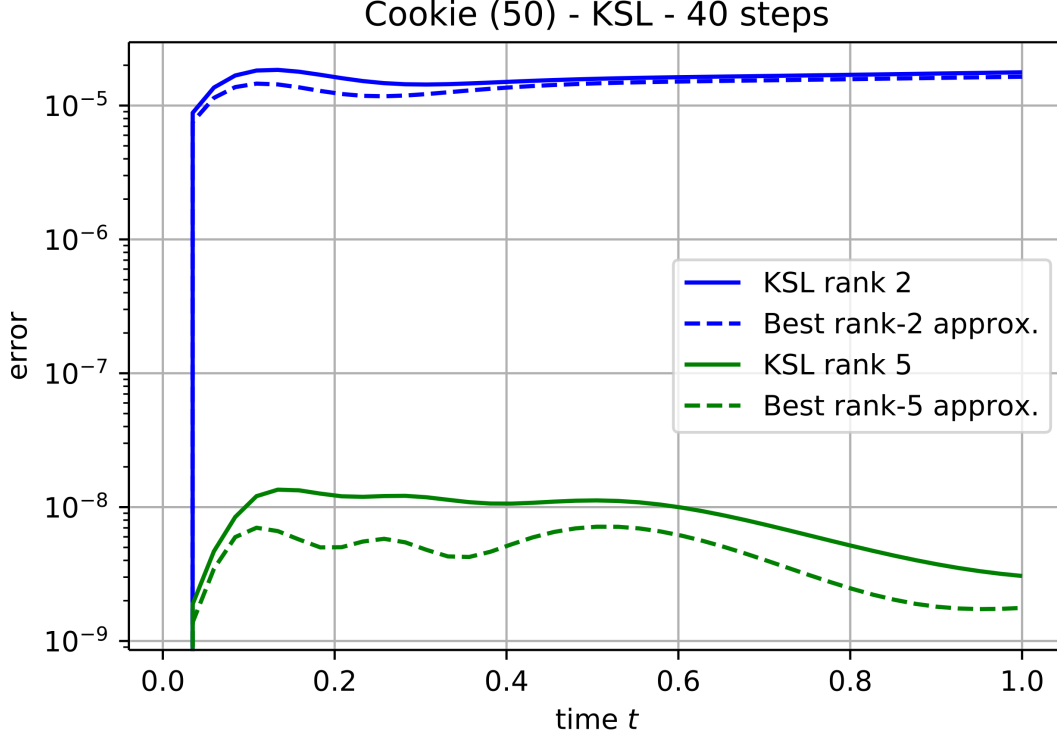


Figure 2.3: The KSL scheme of order 2 applied to the cookie problem with $p = 50$ parameters.

2.3.4 KSL: Cookie problem

The cookie problem is detailed in Section 5.2. For readability, we state again the problem here. The cookie problem is

$$\dot{Y}(t) = A_0 Y(t) + A_1 Y(t) C_1 + \mathbf{b} \mathbf{1}^T, Y(0) = Y_0,$$

where the matrices $A_0, A_1 \in \mathbb{R}^{1580 \times 1580}$ come from a PDE discretization, $\mathbf{b} \in \mathbb{R}^{1580}$ is a constant source and C_1 is a diagonal matrix. The initial value Y_0 is the null matrix, and we again start the algorithm at time $t'_0 = 0.01$ to escape the numerical instabilities due to this initial value. Figure 2.3 is the error made by the KSL scheme of order 2 with two different ranks. As we can see, this scheme works very well since we have already converged to the DLRA solution with only 40 steps. The best rank- r approximation is also here for reference, we again observe the quasi-optimality of the DLRA problem.

2.4 Robust projector splitting integrator

In Section 2.3, we used the order K , S and L to compute the solution. In [2], Ceruti and Lubich proposed very recently an other approach, which leads to a robust integrator. In this method, we solve the K -step and the L -step in parallel and then solve the S -step. Following this idea, we will refer to this algorithm as the K-L:S scheme, or simply the KLS scheme. Its pseudocode is given in Algorithm 2.2.

Algorithm 2.2 Robust projector splitting integrator, or K-L:S scheme.

Require: Initial value $Y_0 = U_0 S_0 V_0^T$, a time discretization $t_0 < t_1 < \dots < t_n$.

for $j=0,1,2,\dots,n-1$ **do**

 In parallel, solve the IVPs :

 1) $\dot{K}(t) = F(K(t)V_j^T)V_j$ with $K(t_j) = U_j S_j$.

 1') Compute QR : $K(t_{j+1}) = U_{j+1}R$, and compute $U_{j+1}^T U_j =: M \in \mathbb{R}^{r \times r}$.

 2) $\dot{L}(t) = F(U_j L(t)^T)^T U_j$ with $L(t_j) = V_j S_j^T$.

 2') Compute QR : $L(t_{j+1}) = V_{j+1}\tilde{R}$, and compute $V_{j+1}^T V_j =: N \in \mathbb{R}^{r \times r}$.

 Solve IVP : $\dot{S}(t) = U_{j+1}^T F(U_{j+1} S(t) V_{j+1}^T) V_{j+1}$ with $S(t_j) = M S_j N^T$.

 Set $S_{j+1} = S(t_{j+1})$.

 Result at step $(j+1)$: $Y_{j+1} = U_{j+1} S_{j+1} V_{j+1}^T$.

end for

Remark 2.3 • *The backward step in Algorithm 2.1, is not present in algorithm 2.2. So this algorithm is easier to apply for parabolic problems.*

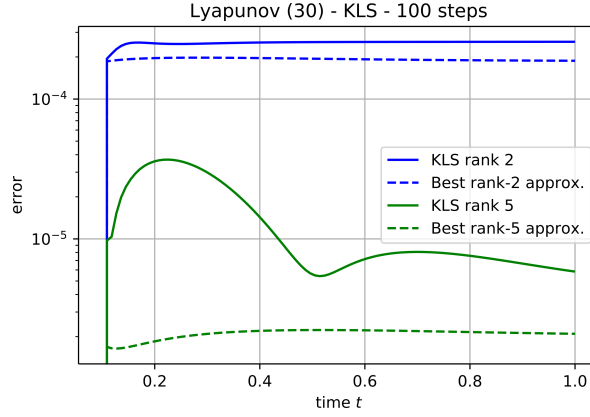
- *Because this scheme is not a conventional splitting scheme, finding the adjoint method is difficult. As a consequence, we do not have the second order scheme.*
- *Ceruti and Lubich also proposed another version of this algorithm in [2]. The idea is to compute the S -step in parallel and then compute $M^{-1} S N^{-1}$. The presence of the inverses is not ideal, so we prefer Algorithm 2.2.*
- *The exactness and robustness properties given in Theorem 2.4 and Theorem 2.5 also hold for this integrator.*
- *Unfortunately, solving K , L and S in order leads to a larger hidden constant compared to the KSL scheme. It was a problem in the numerical experiments because we had to perform a lot of time steps, so we sometimes reduced the size of the problems.*

2.4.1 KLS: Lyapunov

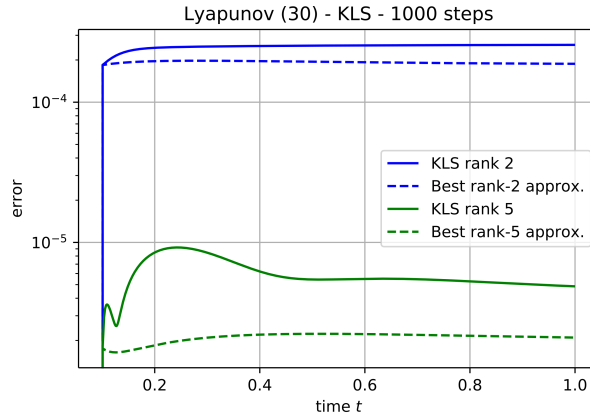
In this section, we test the KLS scheme on the Lyapunov equation. See Section 2.3.3 for a summary or Section 5.1 for details. Figure ?? shows that the KLS scheme has a large hidden constant, so we need a lot of time steps. That's why we reduced the size of the problem to $n = 30$ and used the rank 5 approximation. We obtained a reasonable solution with 4000 time steps.

2.4.2 KLS: Cookie problem

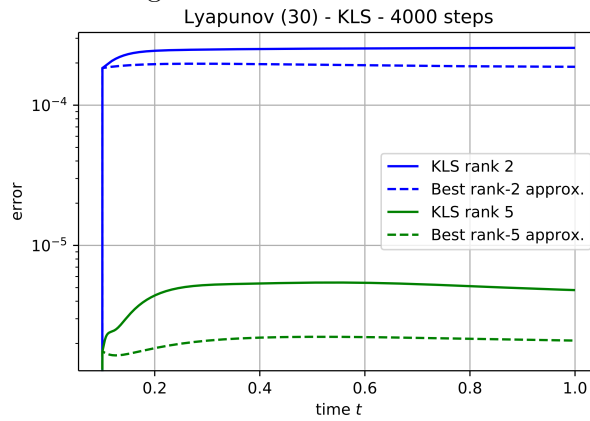
In Figure 2.5a, we applied the KLS scheme to the cookie problem, see Section 2.3.4 for a summary or Section 5.2 for details. With $p = 30$ parameters, and two small ranks, the scheme converges to a satisfactory solution in 1000 steps. However, in Figure 2.5b, we chose higher ranks and we again observe that the KLS scheme suffers from its small order and large hidden constant.



(a) With 100 time steps, the KLS scheme has not converged.

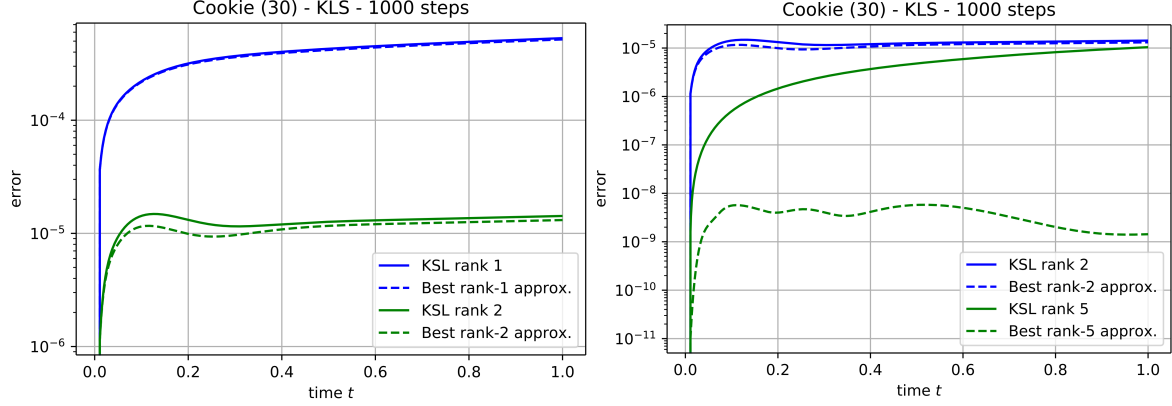


(b) With 1000 time steps, the KLS scheme has not converged.



(c) With 4000 time steps, the KLS scheme has converged.

Figure 2.4: The KLS scheme applied to the Lyapunov problem ($n = 30$). Time starts at $t'_0 = 0.1$. A lot of iterations are necessary because of the large hidden constant and the slow convergence rate.



(a) Coarse rank $r_1 = 1$, fine rank $r_2 = 2$. The scheme has converged to a reasonable solution but the ranks are very small. (b) Coarse rank $r_1 = 1$, fine rank $r_2 = 2$. With higher ranks, the scheme does not converge quickly to the solution.

Figure 2.5: The KLS scheme applied to the cookie problem ($p = 30$).

2.5 Projected Runge–Kutta methods

In [8], Kieri and Vandereycken proposed new integrators for the DLRA ODE (2.4). The main idea is to extend the classical explicit Runge–Kutta methods to the DLRA problem using projectors and retractions onto manifolds to keep the computations small. They called these methods projected Runge–Kutta with s -stages (PRKs).

2.5.1 PRKs: theory

We first recall the classical Runge–Kutta methods. For the autonomous ODE

$$\begin{cases} \dot{Y} = F(Y), \\ Y(0) = Y_0. \end{cases}$$

If one wants to integrate on the time interval $[0, T]$, we discretize $[0, T] = [T_0, T_1] \cup [T_1, T_2] \cup \dots \cup [T_{n-1}, T_n]$ where $T_i - T_{i-1} = h$. Then, the Runge–Kutta method with s -stages (RKs) iterates for $i = 0, 1, 2, \dots, n - 1$:

$$\begin{aligned} \kappa_j &= F \left(Y_i + h \sum_{l=1}^{j-1} a_{jl} \kappa_l \right), \\ Y_{i+1} &= Y_i + h \sum_{j=1}^s b_j \kappa_j, \end{aligned} \tag{2.13}$$

for some coefficients $a_{jl}, b_j \in \mathbb{R}$. The coefficients for the three first orders are

$$\text{RK1} : b_1 = 1,$$

$$\text{RK2} : a_{21} = 1, b_1 = b_2 = \frac{1}{2},$$

$$\text{RK3} : a_{21} = \frac{1}{3}, a_{31} = 0, a_{32} = \frac{2}{3}, b_1 = \frac{1}{4}, b_2 = 0, b_3 = \frac{3}{4}.$$

For higher orders, these coefficients are usually written in a *Butcher Table*.

Now, we can present the Projected Runge–Kutta methods. The key ideas are:

1. Truncate the substeps η_j to the manifold of rank- r matrices, via

$$\mathcal{T}_r : \mathcal{V} \rightarrow \mathcal{M}_r,$$

seen in Section 1.2.

2. Project the derivatives κ_j onto the tangent space $\mathcal{T}_{\eta_j}\mathcal{M}_r$, via

$$\mathcal{P}_{\eta_j}[\cdot] : \mathbb{R}^{m \times n} \rightarrow \mathcal{T}_{\eta_j}\mathcal{M}_r,$$

seen in Section 1.2.

The projected Runge–Kutta method with s -stages iterates for $i = 0, 1, \dots, n-1$:

$$\begin{aligned} \eta_1 &= Y_i, \\ \kappa_j &= \mathcal{P}(\eta_j) [F(\eta_j)], \\ \eta_j &= \mathcal{T}_r \left(Y_i + h \sum_{l=1}^{j-1} a_{jl} \kappa_l \right), \\ Y_{i+1} &= \mathcal{T}_r \left(Y_i + h \sum_{j=1}^s b_j \kappa_j \right). \end{aligned} \tag{2.14}$$

In particular, $\kappa_j \in \mathcal{T}_{\eta_j}\mathcal{M}_r$, $\eta_j \in \mathcal{M}_r$ and $Y_i \in \mathcal{M}_r$. It means by definition that $\text{rank}(\eta_j) = r$ and therefore $\text{rank}(\kappa_j) \leq 2r$ by the definition of the projection given in Section 1.2. Taking the sum, we have that

$$\text{rank} \left(\sum_{j=1}^s b_j \kappa_j \right) \leq 2sr.$$

Finally, in Section 4.2, we show that

$$Y_i + h\mathcal{P}(Y_i)[F(Y_i)] = \mathcal{P}(F_i)[Y_i + hF(Y_i)],$$

so even after adding the term Y_i , we still have

$$\text{rank} \left(Y_i + h \sum_{j=1}^s b_j \kappa_j \right) \leq 2sr.$$

This way, the truncation is applied to matrices of rank at most $2sr$, which makes the computations very efficient.

Kieri and Vandereycken proposed an analysis of the methods. They have shown that, under the assumptions given in Section 2.2, PRK1, PRK2 and PRK3 keep their orders, but this is not true for $s \geq 4$. In our case, we will focus on the projected Euler method (PRK1), i.e.,

$$Y_{i+1} = \mathcal{T}_r(Y_i + h\mathcal{P}(Y_i)[F(Y_i)]). \quad (2.15)$$

In the two following results, ε is the round-off error.

Lemma 2.7 (Local error estimate) *Under the assumptions detailed in Section 2.2, the projected Euler method (2.15) satisfies the error estimate*

$$\|Y_{i+1} - \phi_F^h(Y_i)\|_F \leq Ch(\varepsilon + h), \quad (2.16)$$

where the constant C is independent of h for all $h \leq h_0$ (so there is a step-size restriction).

Theorem 2.8 (Global error estimate) *Under the assumptions detailed in Section 2.2, the projected Euler method (2.15) satisfies the error estimate*

$$\|Y_n - \phi_F^{nh}(Y_0)\|_F \leq C(L, T, h_0, C_{\mathcal{T}})(\delta + \varepsilon + h), \quad (2.17)$$

on the finite time-interval $0 \leq nh \leq T$, for all $0 < h \leq h_0$. The constant C depends only on L, T, h_0 , and $C_{\mathcal{T}}$, but not on h .

Proof The proofs are given in [8]. ■

Remark 2.4 *In Theorem 2.8, the constant $C_{\mathcal{T}}$ comes from the quasi-optimality of the truncated SVD for tensors. More precisely, for a tensor A ,*

$$\|A - \mathcal{T}_{\mathbf{r}}(A)\| \leq C_{\mathcal{T}} \min \{\|A - X\| \mid TT\text{-rank}(X) = \mathbf{r}\},$$

for a given TT -rank $\mathbf{r}$. See [10] and [21] for theory about tensors.

2.5.2 PRKs: algorithm

Algorithm 2.3 is a pseudocode for the projected Runge–Kutta method with s -stages.

Algorithm 2.3 Projected Runge–Kutta with s -stages (PRKs).

Require: Initial value Y_0 , time step h , coefficients a_{ij}, b_j .

```

for  $i = 0, 1, 2, \dots, n - 1$  do
  Define :  $\eta_1 = Y_i$ .
  Compute :  $\kappa_1 = \mathcal{P}(\eta_1)[F(\eta_1)]$ .
  for  $j=0, 1, \dots, s$  do
    if  $j > 0$  then
      Compute :  $\eta_j = \mathcal{T}_r\left(Y_i + h \sum_{l=1}^{j-1} a_{jl}\kappa_l\right)$ .
    end if
    Compute :  $\kappa_j = \mathcal{P}(\eta_j)[F(\eta_j)]$ .
  end for
  Result at step  $i + 1$  :  $Y_{i+1} = \mathcal{T}_r\left(Y_i + h \sum_{j=1}^s b_j\kappa_j\right)$ .
end for
```

Remark 2.5 *If Y_0 is given in factorized form, one should take care of keeping a factorized form along all computations, as explained in Section 1.1.*

2.5.3 PRKs: order of the methods

In this section, we discuss the order of the PRKs methods. We used the Lyapunov problem described in Section 5.1, with final time $t_1 = 0.1$. Figure 2.6a shows the order of classical Runge–Kutta methods. Figure 2.6b shows the order of the projected Runge–Kutta methods, when we use it on the rank-10 approximation. As we can see, PRK1, PRK2 and PRK3 have, respectively, order 1, order 2 and order 3. Figure 2.8 shows the comparison of PRK1, PRK2 and PRK3 on the Lyapunov problem, for a fixed time step.

2.5.4 PRKs: Lyapunov

In this section, we test the projected Runge–Kutta method with 3-stages on the Lyapunov equation. See Section 2.3.3 for a summary or Section 5.1 for details. We used a lot of time steps because there is a step-size restriction in Theorem 2.8. Moreover, the projected Runge–Kutta

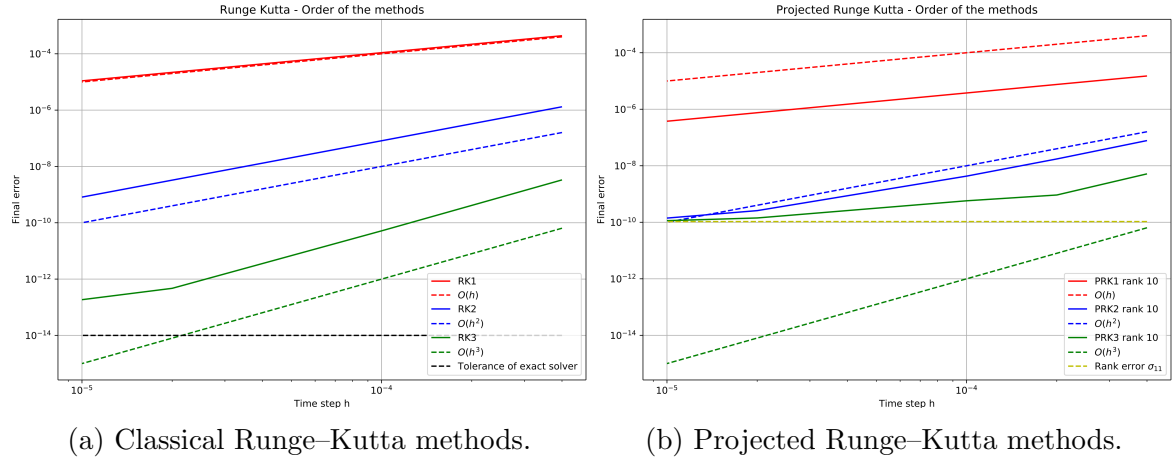


Figure 2.6: Order of (projected) Runge-Kutta methods, on the Lyapunov problem.

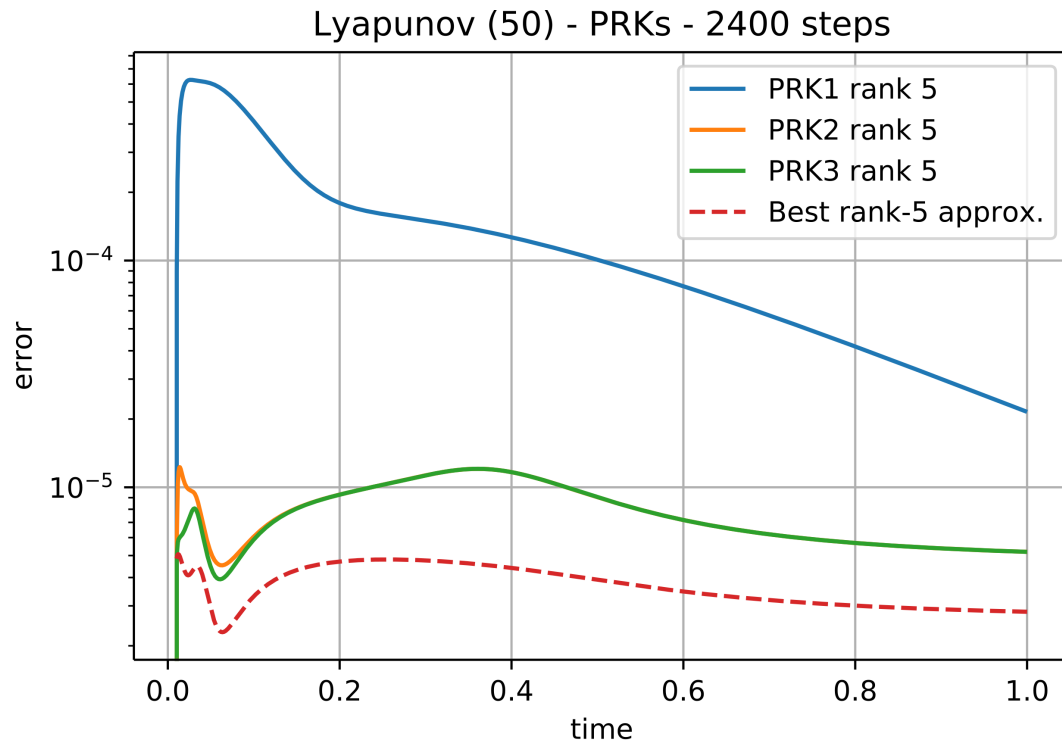


Figure 2.7: Different projected Runge-Kutta methods for rank-5 approximation, applied to the Lyapunov problem ($n = 50$).

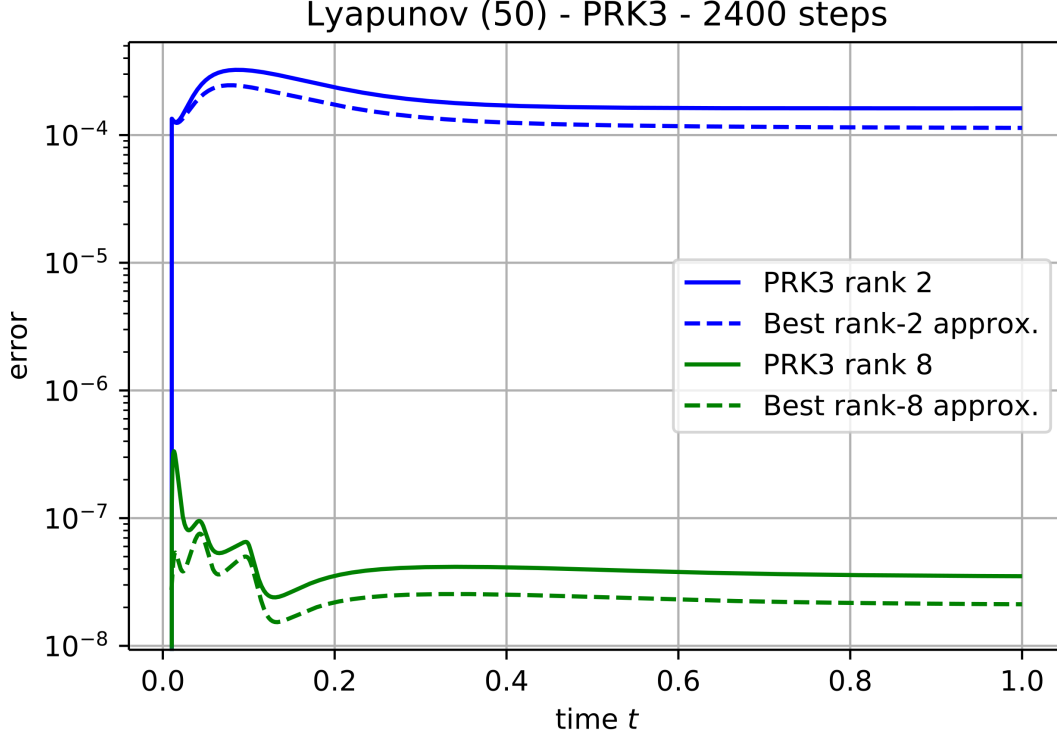


Figure 2.8: The projected Runge–Kutta method with 3-stages applied to the Lyapunov problem ($n = 50$), with two different ranks.

methods proposed by Kieri and Vandereycken are only explicit methods. Since the problem is stiff, we have to be sure to use enough time steps. Figure 2.7 shows the results of the projected Runge–Kutta methods with different stages. If 2400 time steps seems a lot, each iteration can be done very quickly compared to the projector splitting integrators, where the substeps need to be solved accurately.

2.5.5 PRKs: Cookie problem

In this section, we test the projected Runge–Kutta method with 3-stages on the Cookie problem. See Section 2.3.4 for a summary or Section 5.2 for details. In Figure 2.9 we can see that PRK3 has almost converged to the solution of the DLRA problem.

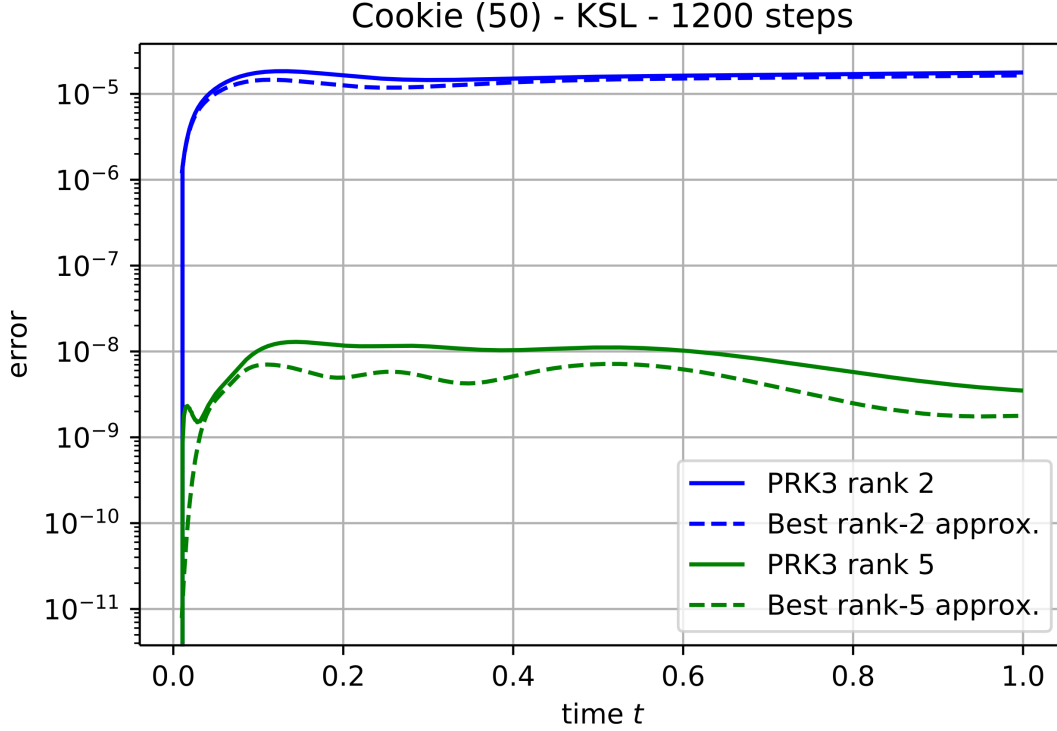


Figure 2.9: The projected Runge–Kutta method with 3-stages applied to the Cookie problem ($p = 50$), with two different ranks.

3 The Parareal algorithm

When we are dealing with time-dependent problems, like the problem (2.1), it seems impossible to compute the solution in parallel. Indeed, the main property of the time is that the solution at time T_n depends on the solution at the time T_{n-1} and so on... In Section 1.1, we have seen that computers have more and more processors and a considerable effort has been made to develop parallel integrators, even for time-dependent problems. One of the most popular is the Parareal algorithm.

The Parareal algorithm was proposed by Lions, Maday and Turninici in [14]. The course given by Martin Gander at the University of Geneva, see [4], presents a large overview of the Parareal algorithm from the heat problem (parabolic) to the transport equation (hyperbolic). After giving the definition of the Parareal algorithm, we give its main properties.

Why the name 'Parareal'? As written by Lions, Maday and Turninici in [14],

We propose in this a scheme which permits the use of a parallel architecture for the time discretization of an evolution partial differential equation. This scheme, based on an Euler discret-

ization, combines coarse solves, and fine independent solves in time, inspired by what is classically done in space. The resulting parallelization is in the time direction, which is in contrast not classical. The main motivation are problems that need to be solved in parallel in real time, hence we call the method 'parareal'.

3.1 Parareal: definition

In this section, we consider the initial value problem,

$$\begin{cases} \dot{\mathbf{u}}(t) = F(\mathbf{u}(t)) & t \in [0, T], \\ \mathbf{u}(0) = \mathbf{u}_0 & \mathbf{u}_0 \in \mathbb{R}^n. \end{cases} \quad (3.1)$$

As usual, we decompose the time interval

$$[0, T] = [T_0, T_1] \cup [T_1, T_2] \cup \dots \cup [T_{N-1}, T_N].$$

Parareal solves (3.1) in parallel with two solvers:

1. The first solver $\underset{T_n \rightarrow T_{n+1}}{G}(u_n)$ computes a coarse¹ (and cheap) approximation of the solution at time T_{n+1} using the initial condition u_n at time T_n . (This compact notation was proposed by Thibaut Lunet.)
2. The second solver $\underset{T_n \rightarrow T_{n+1}}{F}(u_n)$ computes a fine (and expensive) approximation of the solution at time T_{n+1} using the initial condition u_n at time T_n .

The idea is to use the fine and expensive solver in parallel whereas the coarse and cheap solver propagates the solution over the time. In other words, the coarse solver should capture the big picture of the solution and the fine solver should capture the details. The Parareal algorithm iterates at first

$$\begin{cases} U_0^0 = U(0), \\ U_n^0 = \underset{T_{n-1} \rightarrow T_n}{G}(U_{n-1}^0), \text{ for } n = 1, \dots, N. \end{cases} \quad (3.2)$$

Then, for $k = 0, 1, 2, \dots, K$

$$\begin{cases} U_0^{k+1} = U(0), \\ U_n^{k+1} = \underset{T_{n-1} \rightarrow T_n}{F}(U_{n-1}^k) + \underset{T_{n-1} \rightarrow T_n}{G}(U_{n-1}^{k+1}) - \underset{T_{n-1} \rightarrow T_n}{G}(U_{n-1}^k), \end{cases} \quad (3.3)$$

¹We use letter G for "Grossier" in French, which means coarse.

for $n = 1, \dots, N$.

Remark 3.1 *As we can see in the iteration above, we need to use the coarse solver sequentially, which is not a problem since it is cheap to compute. The computation of the fine solution is not cheap but can be done in parallel.*

The pseudocode of the Parareal algorithm is given in Algorithm 3.1.

Algorithm 3.1 Parareal algorithm.

Require: Initial value U_0 , a coarse solver G , a fine solver F .

Define : $U_0^0 = U_0$.

for $n = 1, 2, \dots, N$ **do**

 Compute : $U_n^0 = \underset{T_{n-1} \rightarrow T_n}{G} (U_{n-1}^0)$.

end for

for $k = 0, 1, \dots, K$ **do**

 Define : $U_0^{k+1} = U_0$.

for $n = 1, 2, \dots, N$ **do**

 In parallel, compute : $\underset{T_{n-1} \rightarrow T_n}{F} (U_{n-1}^k)$.

 Compute : $U_n^{k+1} = \underset{T_{n-1} \rightarrow T_n}{F} (U_{n-1}^k) + \underset{T_{n-1} \rightarrow T_n}{G} (U_{n-1}^{k+1}) - \underset{T_{n-1} \rightarrow T_n}{G} (U_{n-1}^k)$.

end for

end for

3.2 Parareal: properties

The first important property of Parareal is the Finite Step Convergence.

Theorem 3.1 (Finite Step Convergence) *Assume that the fine solver has a group property, that is,*

$$\forall t_0, t_1, t_2 \in [0, T], \underset{t_0 \rightarrow t_2}{F} = \underset{t_1 \rightarrow t_2}{F} \circ \underset{t_0 \rightarrow t_1}{F}.$$

The parareal algorithm 3.1 has the property that

$$\text{If } k \geq n, U_n^k = \underset{0 \rightarrow T_n}{F} (U_0),$$

i.e. U_n^k coincides with the fine approximation from iteration index $k = n$.

Proof The proof is by induction.

Initialisation: For $n = 0, U_0^k = U_0$, so this is true by definition.

Induction: Suppose that

$$U_n^k = \underset{0 \rightarrow T_n}{F} (U_0) \text{ for } k \geq n.$$

Because $k \geq n \implies k + 1 \geq n$, we have

$$U_n^k = U_n^{k+1} = F_{0 \rightarrow T_n}(U_0). \quad (3.4)$$

Then, by the definition of the parareal iteration (3.3)

$$\begin{aligned} U_{n+1}^{k+1} &= F_{T_n \rightarrow T_{n+1}}(U_n^k) + G_{T_n \rightarrow T_{n+1}}(U_n^{k+1}) - G_{T_n \rightarrow T_{n+1}}(U_n^k) \\ &= F_{T_n \rightarrow T_{n+1}}(U_n^k) \\ &= F_{T_n \rightarrow T_{n+1}}\left(F_{0 \rightarrow T_n}(U_0)\right) \\ &= F_{0 \rightarrow T_{n+1}}(U_0), \end{aligned}$$

where the second equality holds because of (3.4), so the two terms cancelled. ■

Remark 3.2 • *The assumption made in Theorem 3.1 is verified if the fine solver is the exact solver. It is also true if the fine solver is a splitting method.*

- *A direct consequence of Theorem 3.1 is that we can reduce the number of computations in Algorithm 3.1.*
- *If Finite Step Convergence is a good property on paper, it is not useful in practice since we could have computed the F solver immediately.*

Another interesting property of the Parareal algorithm is the convergence even for $T \rightarrow \infty$ when applied to *Dahlquist's test equation*

$$\begin{cases} \dot{u}(t) = \lambda u(t), & \text{where } \lambda \in \mathbb{C}; \\ u(0) = u_0. \end{cases} \quad (3.5)$$

Theorem 3.2 (Bound for Dahlquist's test equation) *Let the fine solver F be the exact solution, that is,*

$$F_{t_0 \rightarrow t_1}(v) = v e^{\lambda(t_1 - t_0)}.$$

Let R be the stability function of the coarse solver. Assume that $\operatorname{Re}(\lambda) \leq 0$ such that the solution of Dahlquist's test equation stays bounded and $|R(\lambda h)| < 1$. Then, for an arbitrary initialisation, the Parareal algorithm satisfies the linear error bound

$$\sup_{n>0} |u(T_n) - U_n^k| \leq \left(\frac{|e^{\lambda h} - R(\lambda h)|}{1 - |R(\lambda h)|} \right)^k \sup_{n>0} |u(T_n) - U_n^0|.$$

Proof The proof is given in [4]. ■

In other words, the method is consistent since $R(z) = 1 + z + \dots$, and therefore

$$e^{\lambda h} - R(\lambda h) = o(\lambda h).$$

Remark 3.3 The error bound in Theorem 3.2 does neither depend on T_n nor on T . In other words, we can obtain a contraction even when $T \rightarrow \infty$.

When the Parareal algorithm is applied to the heat equation

$$\begin{cases} \dot{u} = \Delta u & \text{in } \Omega \times (0, T], \\ u(\mathbf{x}, 0) = u_0(\mathbf{x}), \end{cases} \quad (3.6)$$

we have the following error bound.

Theorem 3.3 (Superlinear bound for the heat equation) *Let the fine solver F be the exact solver. Let the coarse solver G be an A_0 -stable method, i.e., $|e^z - R(z)| \leq \gamma_s, \forall z \in \mathbb{R}_-$. Then, the Parareal algorithm applied to (3.6) verifies*

$$\max_{1 \leq n \leq N} \|u(\cdot, T_n) - U_n^k\|_2 \leq \frac{\gamma_s^k}{k!} C(u, N, U_n^0) \prod_{l=1}^k (N - l),$$

where γ_s is a constant depending only on the coarse solver G and

$$C(u, N, U_n^0) = \sqrt{\sum_{\omega}^{\infty} \max_{1 \leq n \leq N} \left| \hat{u}(T_n, \omega) - \hat{U}_n^0(\omega) \right|^2},$$

with $\hat{u}$ and $\hat{U}_n^0$ the Fourier coefficients of u and U_n^0 .

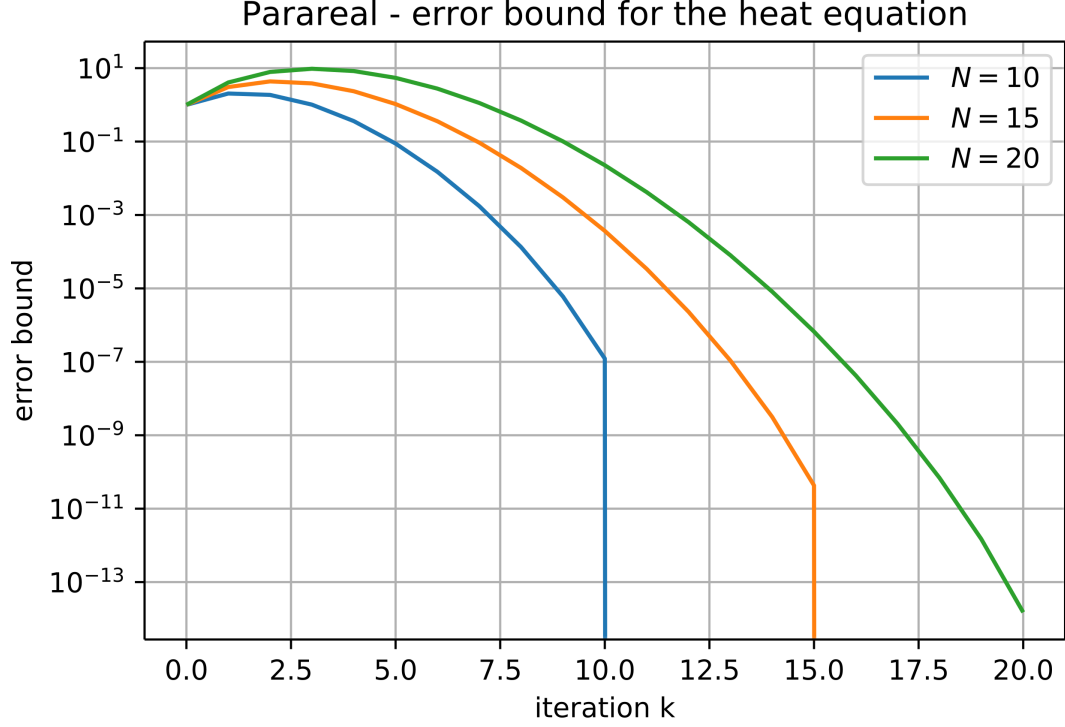


Figure 3.1: Bound for the Parareal algorithm applied to the heat problem, seen in Theorem 3.3. Different values of N and $C(u, N, U_0^N) = 1$. The convergence of Parareal for parabolic problems is good.

Proof The proof is given in [4]. ■

If we choose the coarse solver G as implicit Euler, in the worst case we have $\gamma_s \approx 0.2036321888$. Figure 3.1 shows the bound for different value of N and $C(u, N, U_n^0) = 1$. There also exists a superlinear bound on bounded time intervals. More generally, the Parareal algorithm works well on parabolic problems and we can get a speed-up between 5 and 10.

When the Parareal algorithm is applied to the transport equation

$$\begin{cases} \dot{u} + a\partial_x u = 0 & \text{in } \Omega \times (0, T], \\ u(\mathbf{x}, 0) = u_0(\mathbf{x}), \end{cases} \quad (3.7)$$

we have the following error bound.

Theorem 3.4 (Superlinear bound for the transport equation) *Let the fine solver F be the exact solver and the coarse solver G verifying $|R(z)| < 1$ for all z on the imaginary axes. Then, the Parareal al-*

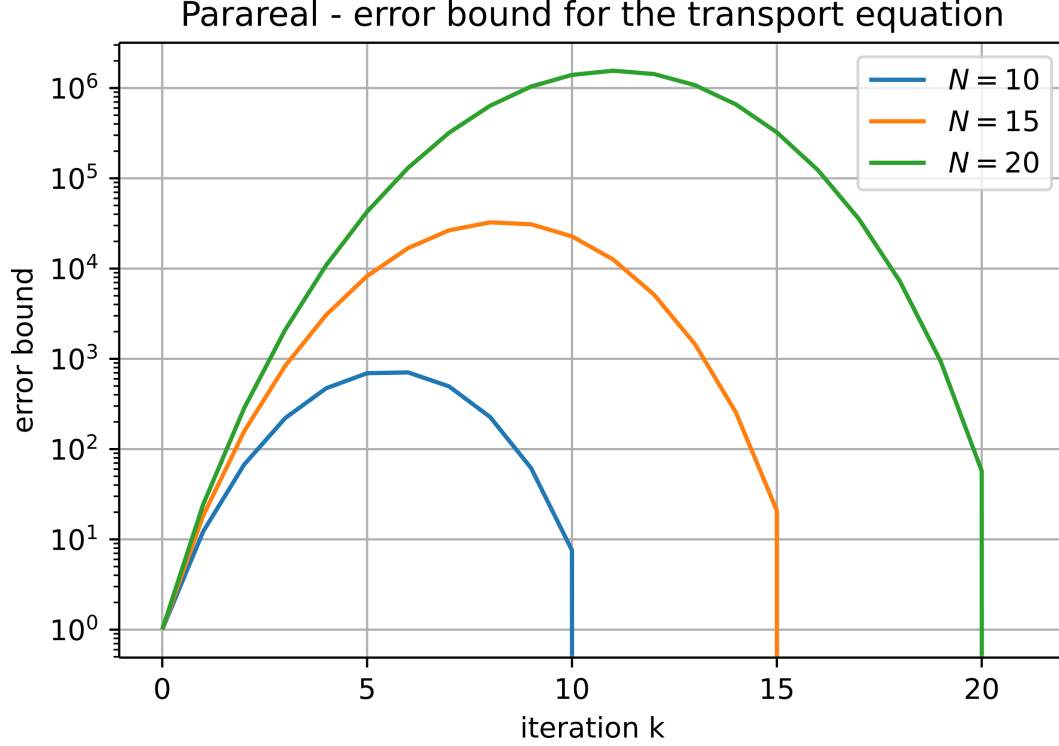


Figure 3.2: Bound for the Parareal algorithm applied to the transport equation, seen in Theorem 3.4. Different values of N and $C(u, N, U_0^N) = 1$. The convergence of Parareal for hyperbolic problems is bad.

gorithm applied to (3.7) verifies

$$\max_{1 \leq n \leq N} \|u(\cdot, T_n) - U_n^k\|_2 \leq \frac{\alpha_s^k}{k!} C(u, N, U_n^0) \prod_{l=1}^k (N - l),$$

where α_s depends only on the coarse solver G and $C(u, N, U_n^0)$ is similar to Theorem 3.3 with the transport equation.

Proof The proof is given in [4]. ■

If we choose the coarse solver G as implicit Euler, in the worst case we have $\alpha_s \approx 1.224353426$. Figure 3.2 shows the bound for different values of N and $C(u, N, U_n^0) = 1$. For this value of α_s , this bound isn't enough to prove the convergence of the Parareal algorithm. On numerical experiments, we can observe that the Parareal algorithm does not necessarily converge before $k = N$, which means no speed-up. More generally for hyperbolic problems, the Parareal algorithm does not work very well, i.e., we can't expect any speed-up from this algorithm.

4 Parallel time integration of DLRA

4.1 A new interpretation of Parareal: Multirank Parareal

In Section 3, we have given the Parareal algorithm for the initial value problem (3.1). How can we extend the Parareal algorithm to the DLRA problem?

The most intuitive idea is to use a cheap integrator of DLRA, for example the KSL scheme with a large time step h . The fine integrator would be then the KSL scheme with a small time step h . We can also use methods of different orders, like PRK1 and PRK3. This approach has two problems.

1. If the integrator has a step size restriction, the coarse solver may become too expensive. In this case, the algorithm is not usable in practice.
2. The coarse solver does not need as much precision as the fine solver. In the context of DLRA, the precision is determined by the rank. Hence, it seems sensible to use a different rank for the coarse solver and the fine solver.

As far as we know, this combination of Parareal and DLRA is completely new. The idea is to solve in parallel

$$\begin{cases} \dot{A}(t) = F(A(t)) \text{ where } t \in [0, T], \\ A(0) = A_0. \end{cases} \quad (4.1)$$

with a coarse rank r_1 and a fine rank r_2 such that $r_1 < r_2$. Then, the coarse solver $G^{r_1} = G \circ \mathcal{T}_{r_1}$ first truncates the input to the rank r_1 and then solves the problem

$$\begin{cases} \dot{W}(t) = \mathcal{P}(W(t)) [F(W(t))], \\ W(t_0) = W_0 \in \mathcal{M}_{r_1}, \end{cases} \quad (4.2)$$

where the projection $\mathcal{P}(W(t)) [F(W(t))]$ is onto the tangent space $\mathcal{T}_{W(t)} \mathcal{M}_{r_1}$. The coarse solver should be precise enough to have

$$G_{0 \rightarrow t}^{r_1}(A_0) \approx A_{r_1}(t).$$

Similarly, the fine solver $F^{r_2} = F \circ \mathcal{T}_{r_2}$ first truncates the input to the rank r_2 and then solves the problem

$$\begin{cases} \dot{Z}(t) = \mathcal{P}(Z(t)) [F(Z(t))], \\ Z(t_0) = Z_0 \in \mathcal{M}_{r_2}, \end{cases} \quad (4.3)$$

where the projection $\mathcal{P}(Z(t)) [F(Z(t))]$ is onto the tangent space $\mathcal{T}_{Z(t)} \mathcal{M}_{r_2}$. The fine solver should be precise enough to have

$$F^{r_2}_{0 \rightarrow t}(A_0) \approx A_{r_2}(t).$$

Because we use two different ranks, we called this method *Multirank Parareal*.

Remark 4.1 *Here, we note that this Multirank Parareal method is different from the original Parareal method. The two problems solved by the coarse and the fine solvers are different. However, the original problem they both try to solve is the same, that is, the IVP (4.1). This also means that there does not already exist an analysis for this method.*

Taking the ranks $r_1 < r_2$ makes the coarse solver fundamentally coarser and cheaper than the fine solver. Indeed, r_1 is (very) small so problem (4.2) is also small and cheap. Moreover, the precision we can reach is limited to $\varepsilon_1 = \sigma_{r_1+1}$ (in the spectral norm). It means that the step size h can be large, or the coarse integrator can have a small order, making it even cheaper. An efficient computation would take advantage of this property by optimizing the step size for the coarse and the fine solvers according to their respective ranks r_1, r_2 . In Section 5, we will observe that, the convergence of this new Parareal algorithm depends on the ranks. In particular, the convergence is better when the difference $r_2 - r_1$ is small. If the difference $r_2 - r_1$ is too large, it seems sensible that taking intermediate ranks between r_2 and r_1 would improve the convergence rate.

An interesting way to take intermediate ranks would be to use a multi-level algorithm. It is possible to extend the Parareal algorithm to multilevel, see [5]. This extension uses multiple solvers. In our case, for n levels, we would choose many ranks $r_1 < r_2 < \dots < r_n$. As before for two levels, we can choose the ranks and the step sizes such that the cost of the computation at each level is optimized. However, in this thesis, we have not explored this further and leave it for future work.

4.2 Analysis

In this section, we define the fine solver as the exact solver of the ODE (2.1). We will also need the assumptions given in Section 2.2. Moreover, we will assume that the function of the ODE given in (2.1) is continuously differentiable, i.e.,

$$F \in \mathcal{C}^\infty \text{ so, } F(u+h) = F(u) + F'(u)h + \frac{F^{(2)}(u)}{2!}h^2 + \frac{F^{(3)}(u)}{3!}h^3 + \dots.$$

The usual notation of the flow $\phi_F^h(u)$ is the exact solution at time h of the initial value problem

$$\begin{cases} \dot{y} = F(y), \\ y(0) = u. \end{cases}$$

Remark 4.2 (Expansion of the flow) *Since we assumed $F \in \mathcal{C}^\infty$, the flow can be expanded in a Taylor series,*

$$\begin{aligned} \phi_F^h(u) &= u + F(u)h + \frac{1}{2}(F \circ y)'(0)h^2 + \frac{1}{6}(F \circ y)''(0)h^3 + \dots \\ &= u + F(u)h + c_2(u)h^2 + c_3(u)h^3 + \dots, \end{aligned}$$

where c_j for $j = 2, 3, \dots$, are continuously differentiable functions.

Lemma 4.1 *Let $\mathcal{T}_r : \mathbb{R}^{m \times n} \rightarrow \mathcal{M}_r$ be the retraction to the rank- r manifold of matrices. For all $v \in \mathcal{M}_r$ and h small enough, we have*

$$\begin{aligned} \mathcal{T}_r(v + h\mathcal{P}(v)[F(v)]) &= v + h\mathcal{P}(v)[F(v)] + \tilde{c}_2(F(v))h^2 + \tilde{c}_3(F(v))h^3 + \dots \\ &= v + hF(v) - h\mathcal{P}^\perp(v)[F(v)] + \tilde{c}_2(F(v))h^2 + \dots, \end{aligned} \tag{4.4}$$

where $\tilde{c}_j$ are continuously differentiable functions.

Proof The proof is by Taylor expansion,

$$\mathcal{T}_r(v + h\mathcal{P}(v)[F(v)]) = \mathcal{T}_r(v) + h\mathcal{T}_r'(v)[\mathcal{P}(v)[F(v)]] + \tilde{c}_2(v)h^2 + \dots. \tag{4.5}$$

From [13, Lemma 2.1], the first order derivative of the retraction evaluated at v is the projection onto $\mathcal{T}_v\mathcal{M}_r$,

$$\mathcal{T}_r'(v)[H] = \mathcal{P}(v)[H]. \tag{4.6}$$

Inserting (4.6) into (4.5) and using that $v \in \mathcal{M}_r$ we get

$$\begin{aligned}\mathcal{T}_r(v + h\mathcal{P}(v)[F(v)]) &= v + h\mathcal{P}(v)[\mathcal{P}(v)[F(v)]] + \tilde{c}_2(F(v))h^2 + \dots \\ &= v + h\mathcal{P}(v)[F(v)] + \tilde{c}_2(F(v))h^2 + \dots.\end{aligned}$$

Finally, the second equality in the lemma is obtained by using

$$\mathcal{P}(v)[F(v)] = I - \mathcal{P}^\perp(v)[F(v)].$$

■

Lemma 4.2 *Let $\beta \geq 1$ and $0 < \alpha < 1$. Assume $e_0^k = 0$ and also that $\forall n, k \in \mathbb{N}, e_n^k \in \mathbb{R}$ verifies the relations*

$$\begin{cases} e_{n+1}^{k+1} = \alpha e_n^k + \beta e_n^{k+1} + \kappa, \\ e_{n+1}^0 = \beta e_n^0 + \gamma. \end{cases}$$

Then, we have that

$$\begin{cases} e_n^k \leq n\kappa(\beta + \alpha)^{n-1} & \text{if } n \leq k, \\ e_n^k \leq n\kappa(\beta + \alpha)^{n-1} + \alpha^k \gamma \frac{\prod_{l=0}^k (n-l)}{(k+1)!} (\beta + \alpha)^{n-k-1} & \text{if } n \geq k+1. \end{cases}$$

Proof Assume we have the relations

$$e_{n+1}^{k+1} = \alpha e_n^k + \beta e_n^{k+1} + \kappa, \tag{4.7}$$

$$e_{n+1}^0 = \beta e_n^0 + \gamma. \tag{4.8}$$

The goal is to find a bound for e_n^k for all n, k . The technique, already used in [4] when $\kappa = 0$, is to use formal sums to identify the coefficients. This method is called generating functions.

First, by the relation (4.8), we get the formal sum

$$\begin{aligned}\sum_{n=0}^{\infty} e_{n+1}^0 \xi^{n+1} &= \sum_{n=0}^{\infty} (\beta e_n^0 + \gamma) \xi^{n+1} \\ &= \beta \xi \sum_{n=0}^{\infty} e_n^0 \xi^n + \gamma \xi \sum_{n=0}^{\infty} \xi^n.\end{aligned}$$

Defining

$$\rho_0(\xi) := \sum_{n=0}^{\infty} e_n^0,$$

and using that $e_0^0 = 0$, we obtain the relation

$$\rho_0(\xi) = \beta\xi\rho_0(\xi) + \gamma\xi\frac{1}{1-\xi},$$

which lead to the formula

$$\rho_0(\xi) = \frac{\gamma\xi}{(1-\beta\xi)(1-\xi)}. \quad (4.9)$$

Second, by the relation 4.7, we get the formal sum

$$\begin{aligned} \sum_{n=0}^{\infty} e_{n+1}^{k+1} \xi^{n+1} &= \sum_{n=0}^{\infty} (\alpha e_n^k + \beta e_n^{k+1} + \kappa) \xi^{n+1} \\ &= \alpha\xi \sum_{n=0}^{\infty} e_n^k \xi^n + \beta\xi \sum_{n=0}^{\infty} e_n^{k+1} \xi^n + \kappa\xi \sum_{n=0}^{\infty} \xi^n. \end{aligned}$$

Defining

$$\rho_k(\xi) := \sum_{n=0}^{\infty} e_n^k \xi^n,$$

and using that $e_0^k = 0$, we obtain the relation

$$\rho_{k+1}(\xi) = \alpha\xi\rho_k(\xi) + \beta\xi\rho_{k+1}(\xi) + \kappa\xi\frac{1}{1-\xi},$$

which leads to the recursive relation

$$\rho_{k+1}(\xi) = \frac{\alpha\xi\rho_k(\xi)}{1-\beta\xi} + \frac{\kappa\xi}{(1-\xi)(1-\beta\xi)} =: a\rho_k(\xi) + b. \quad (4.10)$$

We finally get

$$\begin{aligned} \rho_k(\xi) &= a\rho_{k-1}(\xi) + b \\ &= \dots \\ &= a^k\rho_0(\xi) + \sum_{j=0}^{k-1} a^j b. \end{aligned}$$

Note that, because we are working with formal sums, each inequality means inequality for each coefficient in the sum. For example,

$$\frac{1}{1-\xi} \leq \frac{1}{1-\beta\xi} \iff 1 + \xi + \xi^2 + \dots \leq 1 + \beta\xi + (\beta\xi)^2 + \dots$$

which is true because $\beta \geq 1$ by assumption.

Since

$$\sum_{j=0}^{k-1} a^j \leq \sum_{j=0}^{\infty} a^j = \frac{1}{1-a},$$

we obtain

$$\rho_k(\xi) \leq a^k \rho_0(\xi) + \frac{b}{1-a}. \quad (4.11)$$

Now, using the definition of a and b ,

$$\begin{aligned} \frac{b}{1-a} &= \frac{\kappa\xi}{(1-\xi)(1-\beta\xi)} \frac{1}{1-\frac{a\xi}{1-\beta\xi}} \\ &= \frac{\kappa\xi}{(1-\xi)} \frac{1}{1-(\beta+\alpha)\xi} \\ &\leq \frac{\kappa\xi}{[1-(\beta+\alpha)\xi]^2}. \end{aligned} \quad (4.12)$$

We need two more bounds,

$$a^k = \left(\frac{\alpha\xi}{1-\beta\xi} \right)^k \leq \left(\frac{\alpha\xi}{1-(\beta+\alpha)\xi} \right)^k, \quad (4.13)$$

$$\rho_0(\xi) = \frac{\gamma\xi}{(1-\xi)(1-\beta\xi)} \leq \frac{\gamma\xi}{[1-(\beta+\alpha)\xi]^2}. \quad (4.14)$$

By inserting (4.12), (4.13) and (4.14) into equation (4.11), we get

$$\rho_k(\xi) \leq \frac{\alpha^k \xi^k \gamma \xi}{[1-(\beta+\alpha)\xi]^{k+2}} + \frac{\kappa\xi}{[1-(\beta+\alpha)\xi]^2}.$$

We need to transform this relation back to a relation between two sums.

By the binomial series, we have

$$\begin{aligned} \frac{1}{[1-(\beta+\alpha)\xi]^{k+2}} &= \sum_{j=0}^{\infty} \binom{k+1+j}{j} [(\beta+\alpha)\xi]^j, \\ \frac{1}{[1-(\beta+\alpha)\xi]^2} &= \sum_{j=0}^{\infty} (j+1) [(\beta+\alpha)\xi]^j, \end{aligned}$$

which directly gives

$$\begin{aligned}
\sum_{n=0}^{\infty} e_n^k \xi^n &\leq \alpha^k \xi^k \gamma \xi \sum_{j=0}^{\infty} \binom{k+1+j}{j} [(\beta + \alpha)\xi]^j + \kappa \xi \sum_{j=0}^{\infty} (j+1) [(\beta + \alpha)\xi]^j \\
&= \alpha^k \gamma \sum_{j=0}^{\infty} \binom{k+1+j}{j} (\beta + \alpha)^j \xi^{j+k+1} + \kappa \sum_{j=0}^{\infty} (j+1) (\beta + \alpha)^j \xi^{j+1} \\
&= \alpha^k \gamma \sum_{n=k+1}^{\infty} \binom{n}{n-k-1} (\beta + \alpha)^{n-k-1} \xi^n + \kappa \sum_{n=1}^{\infty} n (\beta + \alpha)^{n-1} \xi^n \\
&= \sum_{n=1}^k n \kappa (\beta + \alpha)^{n-1} \xi^n \\
&\quad + \sum_{n=k+1}^{\infty} \left[\kappa n (\beta + \alpha)^{n-1} + \alpha^k \gamma \binom{n}{n-k-1} (\beta + \alpha)^{n-k-1} \right] \xi^n.
\end{aligned}$$

Since the inequality above is true for each coefficient, we directly obtain the bound announced in the lemma, i.e.,

$$\begin{cases} e_n^k \leq n \kappa (\beta + \alpha)^{n-1} & \text{if } n \leq k, \\ e_n^k \leq n \kappa (\beta + \alpha)^{n-1} + \alpha^k \gamma \frac{\prod_{l=0}^k (n-l)}{(k+1)!} (\beta + \alpha)^{n-k-1} & \text{if } n \geq k+1. \end{cases}$$

■

Lemma 4.3 *Let the fine solver be the exact solver and let the coarse solver be projected Euler computing the rank- r approximation. Under the assumptions in Section 2.2 and assuming that*

$$\| \mathcal{T}_r^\perp(u) - \mathcal{T}_r^\perp(v) \| \leq C_r \|u - v\|, \text{ for all } u, v \in \mathbb{R}^{m \times n},$$

we have

$$\|u(T_{n+1}) - U_{n+1}^{k+1}\| \leq \alpha \|u(T_n) - U_n^k\| + \beta \|u(T_n) - U_n^{k+1}\| + \kappa, \quad (4.15)$$

where

$$\begin{aligned}
\alpha &= (C_r + c_1 h^2 + (1 + C_r)(c_2 h^2)), \\
\beta &= (1 + C_r)(1 + hL + c_2 h^2), \\
\kappa &= 2h\varepsilon(L + 2),
\end{aligned}$$

for some constants c_1 and c_2 .

Proof

By definition of projected Euler, the coarse solver is

$$G_{T_n \rightarrow T_{n+1}}^r(U) = \mathcal{T}_r(X + h\mathcal{P}(X)[F(X)]), \text{ where } X = \mathcal{T}_r(U).$$

Since we assumed that the fine solver is the exact solution, by definition of the parareal iteration (3.3), we have

$$\begin{aligned} u(T_{n+1}) - U_{n+1}^{k+1} = & + \underset{T_n \rightarrow T_{n+1}}{F}(u(T_n)) - \underset{T_n \rightarrow T_{n+1}}{G^r}(u(T_n)) \\ & - \underset{T_n \rightarrow T_{n+1}}{F}(U_n^k) + \underset{T_n \rightarrow T_{n+1}}{G^r}(U_n^k) \\ & + \underset{T_n \rightarrow T_{n+1}}{G^r}(u(T_n)) - \underset{T_n \rightarrow T_{n+1}}{G^r}(U_n^{k+1}). \end{aligned}$$

By definition of each solver, we have

$$\begin{aligned} u(T_{n+1}) - U_{n+1}^{k+1} = & + \phi_F^h(u(T_n)) - \mathcal{T}_r(V + h\mathcal{P}(V)[F(V)]) \\ & - \phi_F^h(U_n^k) - \mathcal{T}_r(W + h\mathcal{P}(W)[F(W)]) \\ & + \mathcal{T}_r(V + h\mathcal{P}(V)[F(V)]) \\ & - \mathcal{T}_r(Z + h\mathcal{P}(Z)[F(Z)]), \end{aligned} \quad (4.16)$$

where $V = \mathcal{T}_r(u(T_n))$, $W = \mathcal{T}_r(U_n^k)$ and $Z = \mathcal{T}_r(U_n^{k+1})$.

By Lemma 4.1 we can expand for V, W and Z ,

$$\mathcal{T}_r(V + h\mathcal{P}(V)[F(V)]) = V + hF(V) - h\mathcal{P}^\perp(V)[F(V)] + h^2\tilde{c}_2(V) + \dots, \quad (4.17)$$

where $\tilde{c}_j$ are continuously differentiable.

By Remark 4.2, the flow can be expanded as

$$\phi_F^h(y) = y + F(y)h + c_2(y)h^2 + \dots, \quad (4.18)$$

where c_j are continuously differentiable.

Then, we can expand every term of the error iteration (4.16) using the two previous relations (4.17), (4.18). For the first part,

$$\begin{aligned} & \phi_F^h(u(T_n)) - \mathcal{T}_r(V + h\mathcal{P}(V)[F(V)]) = \\ & u(T_n) + hF(u(T_n)) + c_2(u(T_n))h^2 + \dots \\ & - V - hF(V) + h\mathcal{P}^\perp(V)[F(V)] - h^2\tilde{c}_2(V) - \dots. \end{aligned} \quad (4.19)$$

For the second part,

$$\begin{aligned} & \phi_F^h(U_n^k) - \mathcal{T}_r(W + h\mathcal{P}(W)[F(W)]) = \\ & U_n^k + hF(U_n^k) + c_2(U_n^k)h^2 + \dots \\ & - W - hF(W) + h\mathcal{P}^\perp(W)[F(W)] - h^2\tilde{c}_2(W) - \dots. \end{aligned} \quad (4.20)$$

For the third part,

$$\begin{aligned} & \mathcal{T}_r(V + h\mathcal{P}(V)[F(V)]) - \mathcal{T}_r(Z + h\mathcal{P}(Z)[F(Z)]) = \\ & V + hF(V) - h\mathcal{P}^\perp(V)[F(V)] + h^2\tilde{c}_2(V) + \dots \\ & - Z - hF(Z) + h\mathcal{P}^\perp(Z)[F(Z)] - h^2\tilde{c}_2(Z) - \dots \end{aligned} \quad (4.21)$$

The goal is to bound the norm of the error iteration (4.16). Since c_j are continuously differentiable, there exists some constant c_1 such that

$$\|c_2(x)h^2 + \dots - c_2(y)h^2 - \dots\| \leq c_1h^2\|x - y\|,$$

and similar inequalities hold with the functions $\tilde{c}_j$ and some constant c_2 .

Assemble and simplify ((4.19) - (4.20) + (4.21)), and also use the inequality above to obtain

$$\begin{aligned} & \|u(T_{n+1}) - U_{n+1}^{k+1}\| \leq \\ & \|\mathcal{T}_r^\perp(u(T_n)) - \mathcal{T}_r^\perp(U_n^k)\| + h\|F(u(T_n)) - F(V)\| + c_1h^2\|u(T_n) - U_n^k\| \\ & + h\|F(U_n^k) - F(W)\| + c_2h^2\|V - W\| \\ & + \|V - Z\| + h\|F(V) - F(Z)\| + c_2h^2\|V - Z\| \\ & + 2h\|\mathcal{P}^\perp(V)[F(V)]\| + h\|\mathcal{P}^\perp(W)[F(W)]\| + h\|\mathcal{P}^\perp(Z)[F(Z)]\|. \end{aligned} \quad (4.22)$$

Now, we will use several inequalities. The first one is assumption (2.7)

$$\|\mathcal{P}^\perp(V)[F(V)]\| \leq \varepsilon.$$

By the definition of the retraction, we have

$$x - \mathcal{T}_r(x) = \mathcal{T}_r^\perp(x).$$

Using this relation and the definition of V and W ,

$$\begin{aligned} \|V - W\| &= \|\mathcal{T}_r(u(T_n)) - \mathcal{T}_r(U_n^k)\| \leq (1 + C_r)\|u(T_n) - U_n^k\|, \\ \|F(u(T_n)) - F(V)\| &\leq L\|u(T_n) - V\| = L\|\mathcal{T}_r^\perp(u(T_n))\| \leq L\varepsilon, \end{aligned}$$

where we used the assumption

$$\|\mathcal{T}_r^\perp(u) - \mathcal{T}_r^\perp(v)\| \leq C_r\|u - v\| \text{ for all } u, v \in \mathbb{R}^{m \times n},$$

in the first inequality, and the Lipschitz assumption (2.5) in the second inequality. Similar inequalities hold with W and Z .

Finally, we can insert all of these inequalities into (4.22), which leads to

$$\begin{aligned} \|u(T_{n+1}) - U_{n+1}^{k+1}\| &\leq (C_r + c_1 h^2 + (1 + C_r)(c_2 h^2)) \|u(T_n) - U_n^k\| \\ &\quad + (1 + C_r)(1 + hL + c_2 h^2) \|u(T_n) - U_n^{k+1}\| \\ &\quad + h\varepsilon(2L + 4), \end{aligned}$$

which concludes the proof. $\blacksquare$

Lemma 4.4 *Under the same assumptions made in Lemma 4.3, we have*

$$\|u(T_{n+1}) - U_{n+1}^0\| \leq \tilde{\beta} \|u(T_n) - U_n^0\| + \gamma, \quad (4.23)$$

where

$$\begin{aligned} \tilde{\beta} &= 1 + hL, \\ \gamma &= \varepsilon + hL\varepsilon + c_3 h^2. \end{aligned}$$

Proof As in the previous proof, the result is obtained by direct computation. We use the definition of the first iteration of parareal,

$$\begin{aligned} u(T_{n+1}) - U_{n+1}^0 &= \phi_F^h(u(T_n)) - G_{T_n \rightarrow T_{n+1}}^r(U_n^0) \\ &= u(T_n) + hF(u(T_n)) + o(h) \\ &\quad - W - hF(W) + h\mathcal{P}^\perp(W)[F(W)] - o(h). \end{aligned}$$

Taking the norm of both side, we obtain

$$\|u(T_{n+1}) - U_{n+1}^0\| \leq \|u(T_n) - W\| + hL \|u(T_n) - W\| + c_3 h^2.$$

Using that $W = \mathcal{T}_r(U_n^0) = U_n^0 - \mathcal{T}_r^\perp(U_n^0)$, and $\|\mathcal{T}_r^\perp(U_n^0)\| \leq \varepsilon$, we can conclude the proof with

$$\|u(T_{n+1}) - U_{n+1}^0\| \leq (1 + hL) \|u(T_n) - U_n^0\| + (\varepsilon + hL\varepsilon + c_3 h^2). \quad \blacksquare$$

Theorem 4.5 (Error bound of the Multirank parareal algorithm)

Let the fine solver be the exact solver and let the coarse solver be projected Euler computing the rank- r approximation. Under the assumptions in Section 2.2 and assuming that

$$\|\mathcal{T}_r^\perp(u) - \mathcal{T}_r^\perp(v)\| \leq C_r \|u - v\| \text{ for all } u, v \in \mathbb{R}^{m \times n},$$

the new parallel time integrator verifies the error bound

$$\begin{aligned} e_n^k \leq & 2nh\varepsilon(L+2) \left((1+C_r)(1+hL+2c_2h^2) + C_r + c_1h^2 \right)^{n-1} \\ & + (C_r + c_1h^2 + (1+C_r)c_2h^2)^k (\varepsilon + hL\varepsilon + c_3h^2) \frac{\prod_{l=0}^k (n-l)}{(k+1)!} \\ & \times \left((1+C_r)(1+hL+2c_2h^2) + C_r + c_1h^2 \right)^{n-k-1}, \end{aligned}$$

for all $k < n$, with c_1, c_2 and c_3 some constants. If $k \geq n$, $e_n^k = 0$.

Proof Define the error

$$e_n^k := \|u(T_n) - U_n^k\|.$$

By Lemma 4.3, we get the first bound

$$e_{n+1}^{k+1} \leq \alpha e_n^k + \beta e_n^{k+1} + \kappa,$$

where

$$\begin{aligned} \alpha &= (C_r + c_1h^2 + (1+C_r)(c_2h^2)), \\ \beta &= (1+C_r)(1+hL+c_2h^2), \\ \kappa &= 2h\varepsilon(L+2). \end{aligned}$$

By Lemma 4.4, we get the second bound

$$e_{n+1}^0 \leq \tilde{\beta} e_n^0 + \gamma,$$

where $\tilde{\beta} = 1 + hL$ and $\gamma = \varepsilon + hL\varepsilon + c_3h^2$. Observing that

$$\tilde{\beta} = 1 + hL \leq (1+C_r)(1+hL+c_2h^2) = \beta$$

implies also

$$e_{n+1}^0 \leq \beta e_n^0 + \gamma.$$

We conclude by using Lemma 4.2, so

$$e_n^k \leq n\kappa(\beta + \alpha)^{n-1} + \alpha^k \gamma \frac{\prod_{l=0}^k (n-l)}{(k+1)!} (\beta + \alpha)^{n-k-1},$$

and $e_n^k = 0$ for $k \geq n$ comes from Theorem 3.1 of finite step convergence. ■

Corollary 4.6 (Simplification of the bound) *Under the assumptions of Theorem 4.5, the error of Multirank parareal verifies*

$$e_n^k \leq nh\varepsilon \Phi_n(L, h, C_r) + \frac{(nC_r + nh^2\chi)^k}{(k+1)!} ((1+hL)n\varepsilon + c_3nh^2) \tilde{\Phi}(L, h, C_r)^{n-k-1},$$

where $\chi = c_1 + (1 + C_r)c_2$; $\Phi_n(L, h, C_r)$ depends on n, L, h, C_r and $\tilde{\Phi}(L, h, C_r)$ depends on L, h, C_r . See the proof for exact definitions.

Proof We obtain this result directly by Theorem 4.5, where we defined

$$\Phi_n(L, h, C_r) = 2(L+2) \left((1+C_r)(1+hL+2c_2h^2) + C_r + c_1h^2 \right)^{n-1},$$

and

$$\tilde{\Phi}(L, h, C_r) = \left((1+C_r)(1+hL+2c_2h^2) + C_r + c_1h^2 \right),$$

and also used that $\forall l \geq 0, (n-l) \leq n$, so $\prod_{l=0}^k (n-l) \leq n^{k+1}$. ■

Remark 4.3 • *In Theorem 4.5, we used projected Euler as coarse solver. The analysis can be extended to projected Runge–Kutta methods of higher order.*

- *We used the exact solver as fine solver, but the analysis can be extended with few modifications to a projected Runge–Kutta method as fine solver.*
- *It is possible to find a result without the assumption in the theorem. If u and v admit good low-rank approximation, one can use the bound*

$$\| \mathcal{T}_r^\perp(u) - \mathcal{T}_r^\perp(v) \| \leq 2\varepsilon.$$

In this case, we add a term 2ε at each iteration of the algorithm, which means the round-off error is accumulating over the iterations. We end with a bound containing $n\varepsilon$ instead of $nh\varepsilon$.

The assumption

$$\| \mathcal{T}_r^\perp(u) - \mathcal{T}_r^\perp(v) \| \leq C_r \|u - v\| \text{ for all } u, v \in \mathbb{R}^{m \times n},$$

is important and deserve some comments. In Theorem 4.5, we need

$$(C_r + c_1h^2 + (1+C_r)(c_2h^2)) < 1.$$

For a convergence, we thus necessarily need $C_r < 1$. As far as we know, there doesn't exist such theoretical bound. However, we can estimate it with numerical experiments. To do so, we define

$$\begin{aligned} X &= \exp(\text{diag}(0, -1, -2, \dots, -15)), \\ X(h) &= X + hE, \end{aligned}$$

where E is a random matrix normally distributed. Then, one can plot

$$C_r(h) = \frac{\|\mathcal{T}_r^\perp(X) - \mathcal{T}_r^\perp(X(h))\|}{\|X - X(h)\|}.$$

In Figure 4.1, we plotted $C_r(h)$ for several random matrix E , and three different ranks. We can estimate

$$\begin{aligned} C_2 &\approx 0.85, \\ C_5 &\approx 0.70, \\ C_{12} &\approx 0.20. \end{aligned}$$

In particular $C_r < 1$ so the assumption in the theorem is reasonable. Moreover, C_r is decreasing in r which is consistent with the numerical experiments of Section 5.

Remark 4.4 *The difficulty of studying such algorithm comes from the fact that the coarse solver and the fine solver don't solve the same problems. One problem is fundamentally less precise than the other one. In the analysis, it seems that the coarse solver brings an extra error in space at each iteration. The assumption above solves partially this problem. This kind of integrator is quite recent and an analysis is not yet available. In [15] and [11], they propose a similar scheme (different levels of the finite element methods), but with no analysis.*

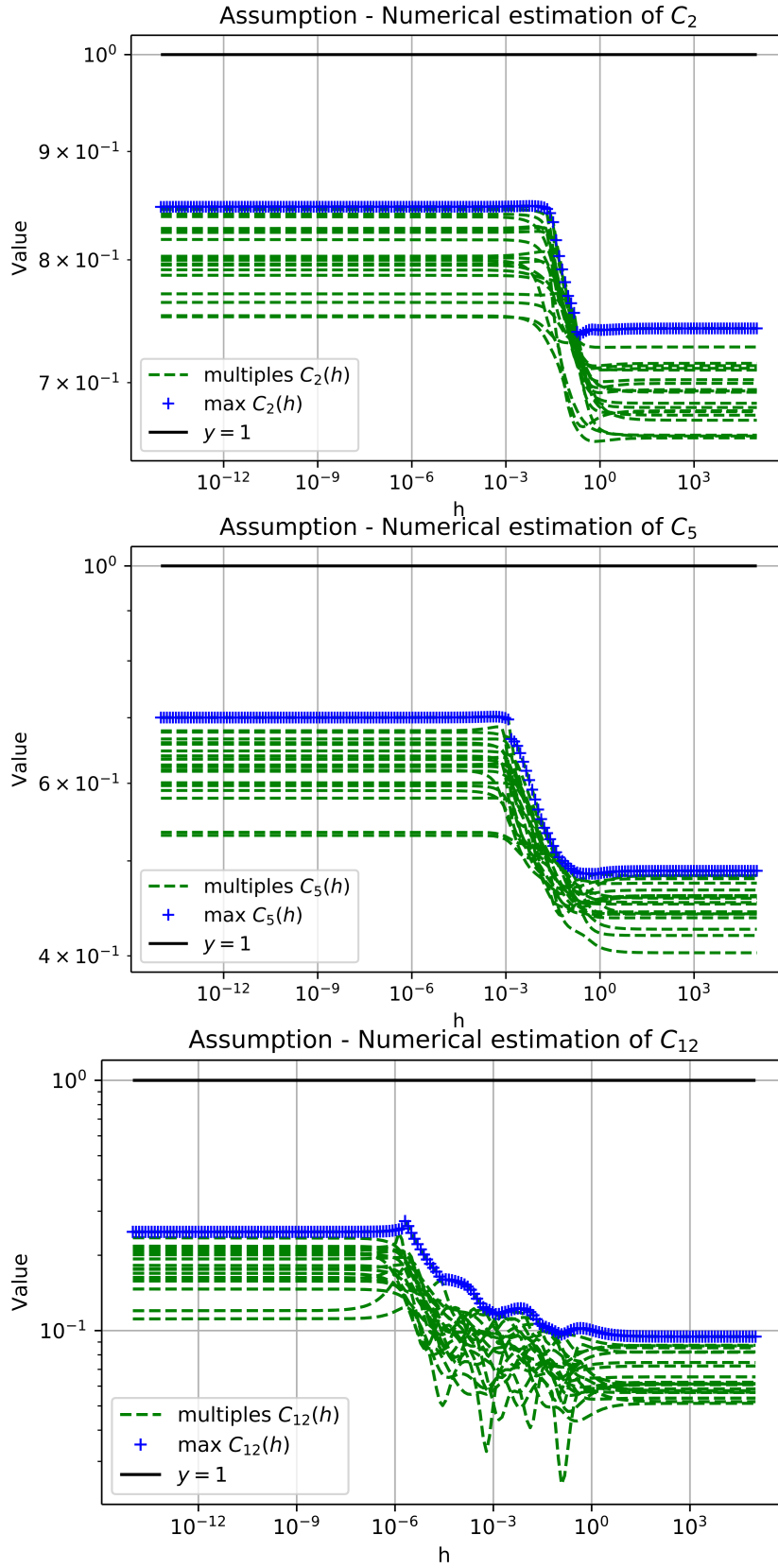


Figure 4.1: Numerical experiments on the assumption made in Theorem 4.5.

5 Numerical experiments

5.1 Lyapunov Equation

The Lyapunov equation is given by

$$\dot{X}(t) = AX(t) + X(t)A + C, \quad (5.1)$$

where $X(t) \in \mathbb{R}^{n \times n}$, $t \in [t_0, t_1]$ and for some matrices $A, C \in \mathbb{R}^{n \times n}$. The initial value is $X(t_0) = X_0$.

5.1.1 Lyapunov: Heat problem

In (5.1), if A is the 1-dimensional Laplacian and C is the source, (5.1) solves a 2-dimensional heat problem. In this case, $X(t)$ is the distribution of the heat over time. In all our numerical experiments, we will use the following settings, and we will refer to this problem as the Lyapunov problem.

- Size of the problem is $n = 50$. Sometimes $n = 30$ for computational reasons, when specified.
- Initial time is $t_0 = 0$ and final time is $t_1 = 1$.
- X_0 is a symmetric random matrix with exponentially decaying singular values, and its rank is $R = 10$.
- $C = \mathbf{u}\mathbf{u}^T$, for a random vector $\mathbf{u} \in \mathbb{R}^{50}$, normally distributed. In particular, the source is rank 1.
- To skip the numerical instabilities due to the non-realistic initial value, we start using the solvers at time $t'_0 = 0.01$, with the exact initial value $X(t'_0)$.

If the initial value $X(t_0) = X_0$ and the source C are low-rank, it can be proven that $X(t)$ admits good low-rank approximation for every time t . Figure 5.1 shows the singular values of the exact solution $X(t)$ for three different times t .

5.1.2 Lyapunov: computing the substeps

In Section 2.3 we derived the KSL scheme. In this scheme, we have to solve three substeps which are three small ODEs. The first possibility

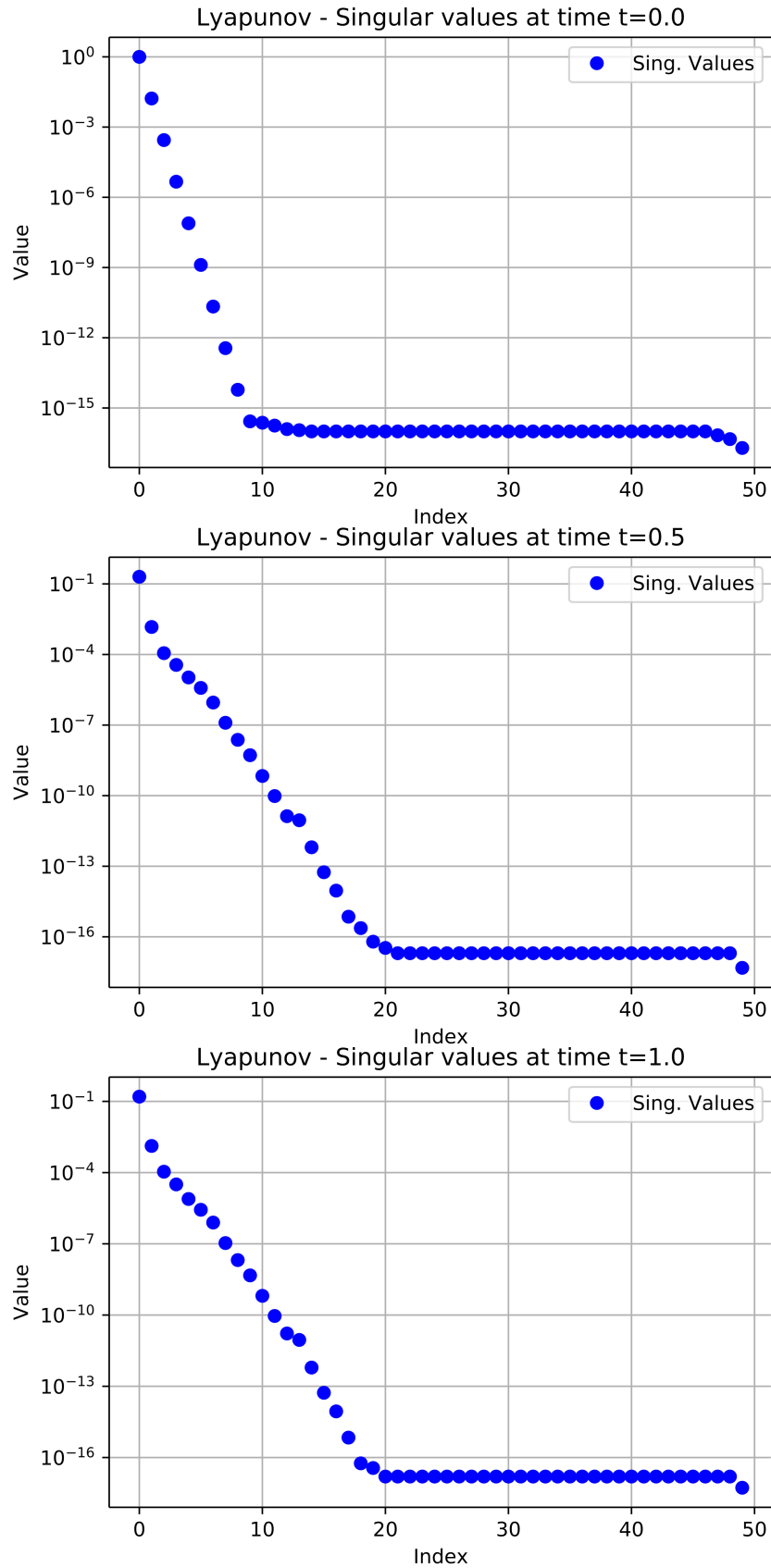


Figure 5.1: Singular values over time of the exact solution of a heat problem with the Lyapunov equation.

to solve these ODEs is to use a classical integrator like a Runge–Kutta method. However, it is sometimes possible to find the closed form solution of each substep. The closed form is more precise, but its computation can be more expensive than using an integrator. One can verify which method is the fastest with numerical experiments. For a better precision, we always used the closed form solutions. We used the following method. Recall the first substep of the KSL scheme is to solve

$$\begin{cases} \dot{K}(t) = F(K(t)V_0^T)V_0, \\ K(0) = U_0S_0. \end{cases}$$

For the Lyapunov equation (5.1), it is

$$\begin{cases} \dot{K}(t) = AK(t) + K(t)V_0^TAV_0 + CV_0, \\ K(0) = U_0S_0. \end{cases} \quad (5.2)$$

We know that for a general problem of the form

$$\begin{cases} \dot{X}(t) = \tilde{A}X + X\tilde{B} + \tilde{C}, \\ X(0) = X_0, \end{cases} \quad (5.3)$$

the explicit solution is given by

$$X(t) = \exp(t\tilde{A})(X_0 + Z)\exp(t\tilde{B}) - Z, \quad (5.4)$$

where Z is the solution of the Sylvester equation $\tilde{A}Z + Z\tilde{B} = \tilde{C}$. One can check this result by simply computing the derivative of $X(t)$. To summary, to compute the solution of the first substep (5.2) we have to:

1. Find Z by solving the Sylvester equation

$$AZ + ZV_0^TAV_0 = CV_0.$$

2. The exact solution of the first substep at time t is given by

$$K(t) = \exp(tA)(U_0S_0 + Z)\exp(tV_0^TAV_0) - Z.$$

Remark 5.1 (*Efficient computing*)

- *Sylvester.* We have to solve the equation $\tilde{A}Z + Z\tilde{B} = \tilde{C}$, where $\tilde{A}$ is a large sparse matrix and $\tilde{B} \in \mathbb{R}^{r \times r}$ is small. By [20] we have to compute the EVD of $B = Q \Lambda Q^{-1}$, which is an $O(r^3)$ cost. Then we solve r sparse linear systems with $(A - \lambda_i I)$.

- *Matrix exponential.* By [1], when A is sparse, it is possible to compute efficiently the action of the matrix exponential. This method is implemented in Python in the package `scipy.sparse`. See also [19], where they use a Krylov method when A is a large sparse matrix. A package is available for MATLAB and Fortran.

The derivation also holds for the two other substeps S and L , with different $\tilde{A}$, $\tilde{B}$ and $\tilde{C}$. With this method we can compute the explicit solution of each substep efficiently. Note that the most expensive step is to solve the Sylvester equation. We always used this method in our experiments.

5.1.3 Lyapunov: Multirank Parareal

In this section, we will apply Multirank Parareal to the Lyapunov problem described in Section 5.1.1. We will try and compare the three integrators seen in Section 2.

The first integrator is the KSL scheme. In Figure 5.2a, we defined the coarse rank $r_1 = 2$ and the fine rank $r_2 = 8$. As we can see, the error starts at the error of the coarse solver and converges quickly to the error of the fine solver'. In 25 iterations, the Multirank Parareal algorithm has converged. If we have access to 60 processors, this means a speed-up of $60/25 = 2.4$.

In Figure 5.2, we compare the Multirank Parareal algorithm for different choices of ranks. As we can see, the convergence rate of the Multirank Parareal algorithm seems to depend on the choice of the coarse rank. In the analysis done in Section 4.2, we have seen that the convergence rate depends on the constant C_r in the assumption

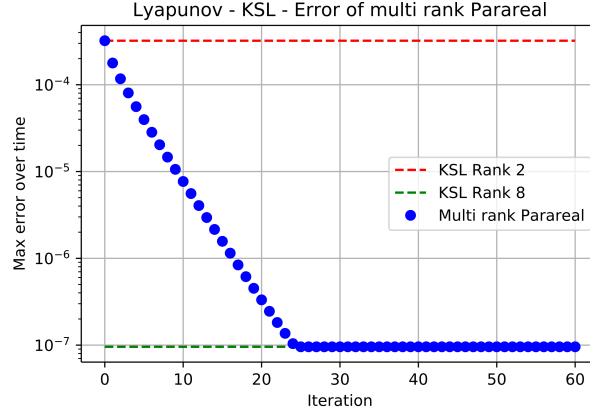
$$\|\mathcal{T}_r^\perp(U) - \mathcal{T}_r^\perp(V)\| \leq C_r \|U - V\|.$$

The second integrator is projected Runge–Kutta of order 3 (PRK3). In Figure 5.3, we applied Multirank Parareal with this integrator. The convergence is very similar to the one with the KSL scheme in Figure 5.2a. This suggests that the convergence of the Multirank Parareal algorithm does not depend on the choice of the integrator.

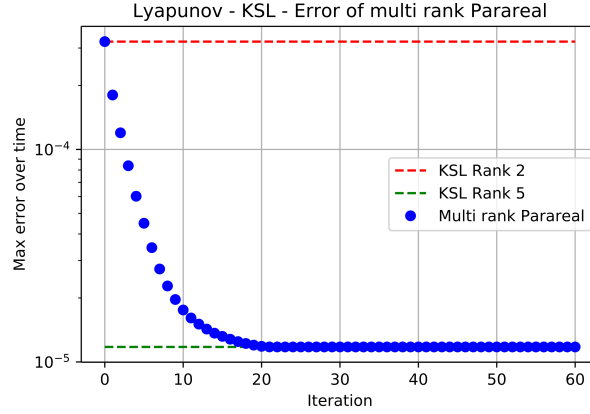
The third and last integrator is the K-L:S scheme, which is more suitable for the heat equation. In Figure 5.4, we applied Multirank Parareal with this integrator. Since this integrator is only of order 1,

we had to reduce the size of the problem to $n = 30$ and take the ranks $r_1 = 2$ and $r_2 = 5$. As we can see, the algorithm converges quickly to the fine solver error. In this case, we observe a speed-up of $40/10 = 4$.

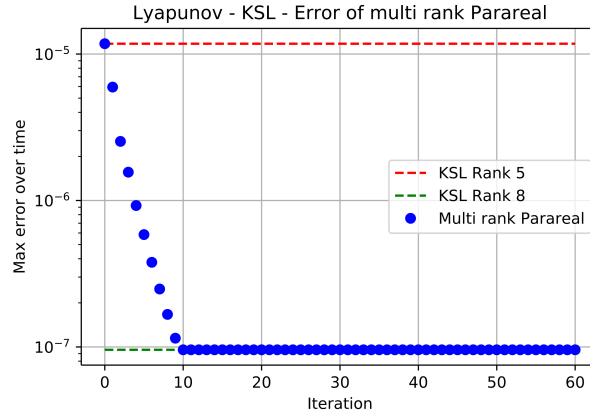
These numerical experiments suggest that Multirank Parareal works well for the heat equation. More generally, it seems that Multirank Parareal could work well on parabolic problems, just like the original Parareal algorithm is proven to do.



(a) Coarse rank $r_1 = 2$, fine rank $r_2 = 8$. The convergence is achieved in 25 iterations. Speed-up = $60/25 = 2.4$.



(b) Coarse rank $r_1 = 2$, fine rank $r_2 = 5$. The convergence is achieved in 20 iterations. Speed-up = $60/20 = 3$.



(c) Coarse rank $r_1 = 5$, fine rank $r_2 = 8$. The convergence is achieved in 10 iterations. Speed-up = $60/10 = 6$.

Figure 5.2: Multirank Parareal applied to the Lyapunov problem ($n = 50$) with the KSL scheme. A higher coarse rank seems to give a better convergence rate.

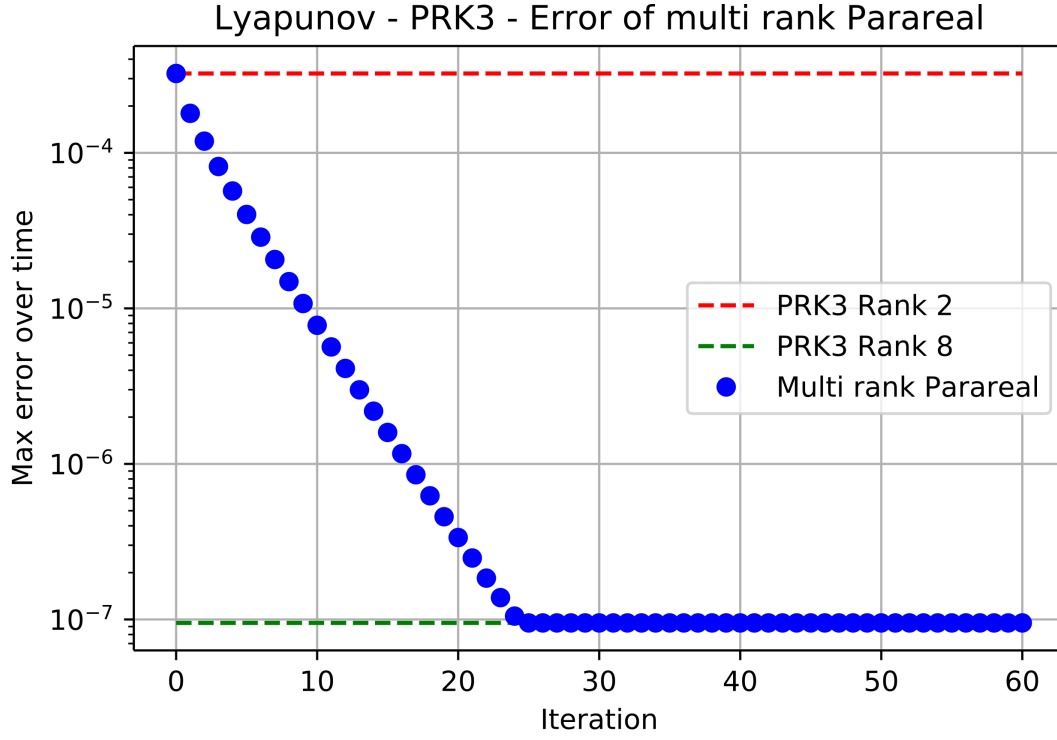


Figure 5.3: Multirank Parareal applied to the Lyapunov problem ($n = 50$) with the PRK3 method. The convergence is very similar to the one with the KSL scheme.

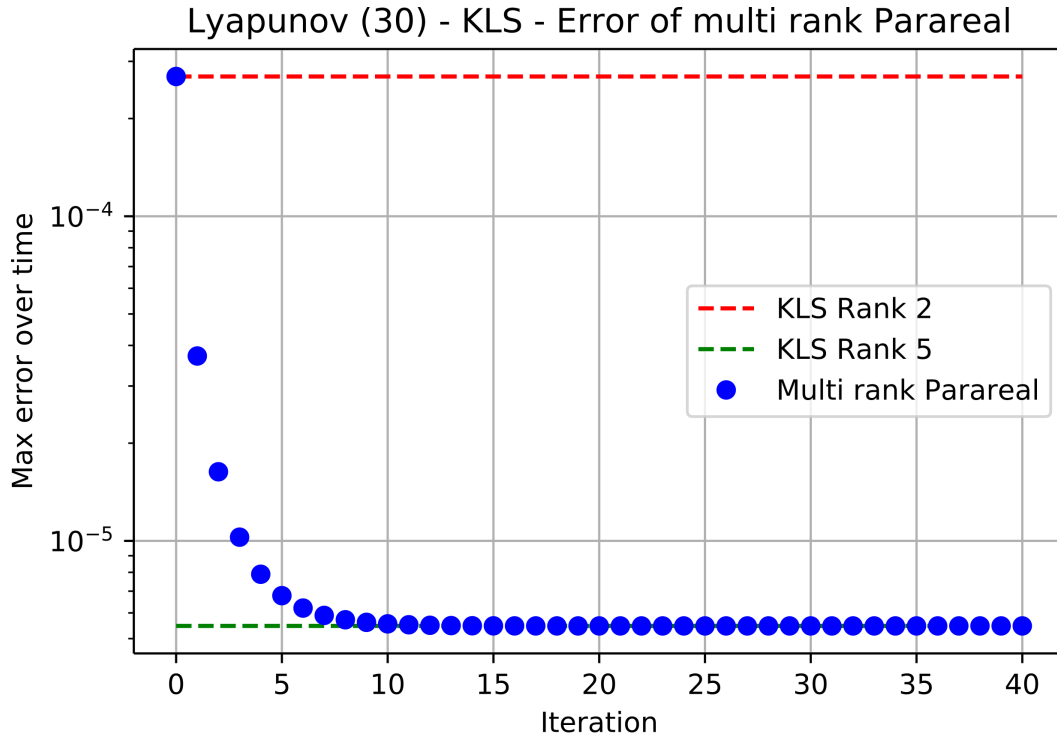


Figure 5.4: Multirank Parareal applied to the Lyapunov problem ($n = 30$) with the K-L:S scheme. Speed-up = $40/10 = 4$.

5.2 Cookie problem

5.2.1 Definition

The cookie problem is a parametric problem based on a discretization of a PDE that models the propagation of heat on a 2-dimensional space containing 4 cookies. The data of this problem comes from [12]. A discretization leads to the ODE problem

$$\mathbf{y}'(t) = -A\mathbf{y}(t) + \mathbf{b} = F(\mathbf{y}(t)), \quad (5.5)$$

where $A \in \mathbb{R}^{1580 \times 1580}$ and $\mathbf{b} \in \mathbb{R}^{1580}$ are given. Since there are four cookies, the matrix A is obtained by assembling

$$A = A_0 + c_1 A_1 + c_2 A_2 + c_3 A_3 + c_4 A_4,$$

where c_i is the heat coefficient associated to the i th cookie represented by A_i . Figure 5.5 shows the triangulation of the problem, where we can see the four cookies.

The parametric problem is to solve the equation of the ODE (5.5) for many c_i in a certain range. In our case, we focus on the first cookie so $c_2 = c_3 = c_4 = 1$ (from now, they are included in A_0). For one parameter we have to solve

$$\dot{\mathbf{y}} = -A\mathbf{y} + \mathbf{b} = -(A_0 + A_1 c_1)\mathbf{y} + \mathbf{b}. \quad (5.6)$$

We want to solve this problem for all c_1 between c_1^1 and c_1^p , i.e.,

$$\begin{aligned} \dot{\mathbf{y}}_1 &= -(A_0 + A_1 c_1^1)\mathbf{y}_1 + \mathbf{b}, \\ &\vdots \\ \dot{\mathbf{y}}_p &= -(A_0 + A_1 c_1^p)\mathbf{y}_p + \mathbf{b}. \end{aligned} \quad (5.7)$$

We can solve all of these problems at once by putting all y_i as the columns of a matrix Y . This leads to the matrix ODE

$$\begin{aligned} \dot{Y} &= [\dot{\mathbf{y}}_1, \dots, \dot{\mathbf{y}}_p] \\ &= [-(A_0 + A_1 c_1^1)\mathbf{y}_1 + \mathbf{b}, \dots, -(A_0 + A_1 c_1^p)\mathbf{y}_p + \mathbf{b}] \\ &= -A_0 Y - A_1 Y C_1 + \mathbf{b} \mathbf{1}^T. \end{aligned}$$

When we mention the cookie problem, we refer to the problem given by the ODE

$$\begin{cases} \dot{Y} = -A_0 Y - A_1 Y C_1 + \mathbf{b} \mathbf{1}^T, \\ Y(0) = Y_0, \end{cases} \quad (5.8)$$

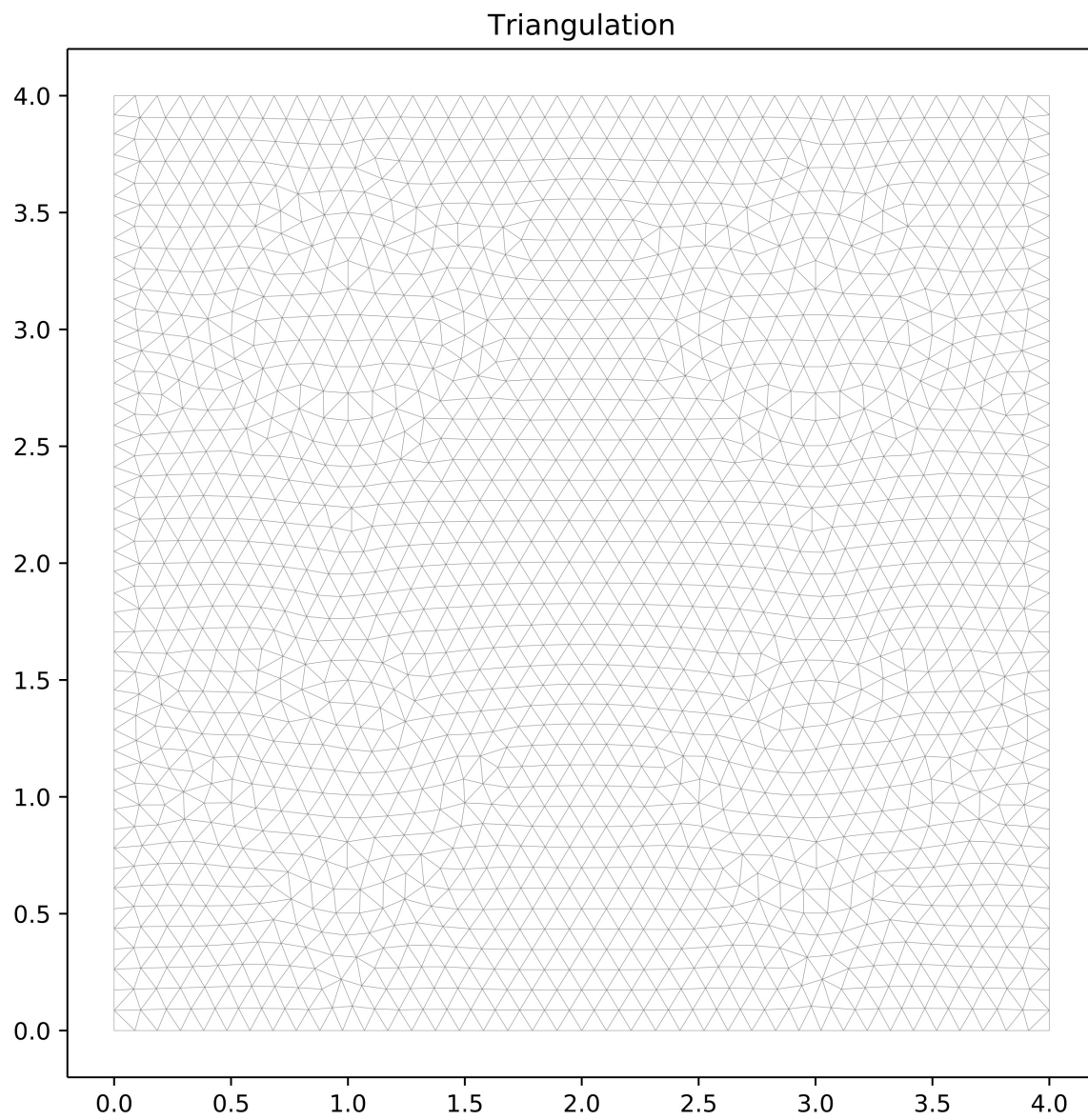


Figure 5.5: Triangulation used in the cookie problem.

where the sparse matrices $A_0, A_1 \in \mathbb{R}^{1580 \times 1580}$ and $b \in \mathbb{R}^{1580}$ are given in [12], and $C = \text{diag}(c_1^1, c_1^2, \dots, c_1^p)$.

Remark 5.2 *It is also possible to derive a similar equation when the four parameters c_1, c_2, c_3, c_4 are varying. In this case, we get a tensor ODE.*

The settings for the numerical experiments of the cookie problem (5.8) are:

- The problem is solved for c_1 linearly distributed between $c_1^1 = 20$ and $c_1^p = 50$.
- The size of the problem is $p = 50$. Sometimes $p = 30$ for computational reasons, when specified.
- The time starts at $t_0 = 0$ and finishes at $t_1 = 1$.
- The initial value is $Y(0) = 0 \in \mathbb{R}^{1580 \times p}$.
- To skip the numerical instabilities due to the non-realistic initial value, we start using the solvers at time $t'_0 = 0.01$, with the exact initial value $X(t'_0)$.

If c_1^1 is close to c_1^2 , the solution $\mathbf{y}_1(t)$ should be close to $\mathbf{y}_2(t)$ and so on. Hence, the matrix $Y(t) \in \mathbb{R}^{1580 \times p}$ should admit a good low-rank approximation. See [12] for a proof. Figure 5.6 shows the singular values of the exact solution $Y(t)$ for three different times t .

5.2.2 Cookie problem: computing the substeps

In Section 2.3 we derived the KSL scheme. In this scheme, we have to solve three substeps which are three small ODEs. In this section, we derive the explicit solution and give a method to solve each substep efficiently. Recall the first substep is to solve

$$\begin{cases} \dot{K}(t) = F(K(t)V_0^T)V_0, \\ K(0) = U_0S_0. \end{cases}$$

For the cookie problem (5.8), this leads to

$$\begin{cases} \dot{K}(t) = -A_0K(t) - A_1K(t)V_0^TC_1V_0 + b1^TV_0, \\ K(0) = U_0S_0. \end{cases} \quad (5.9)$$

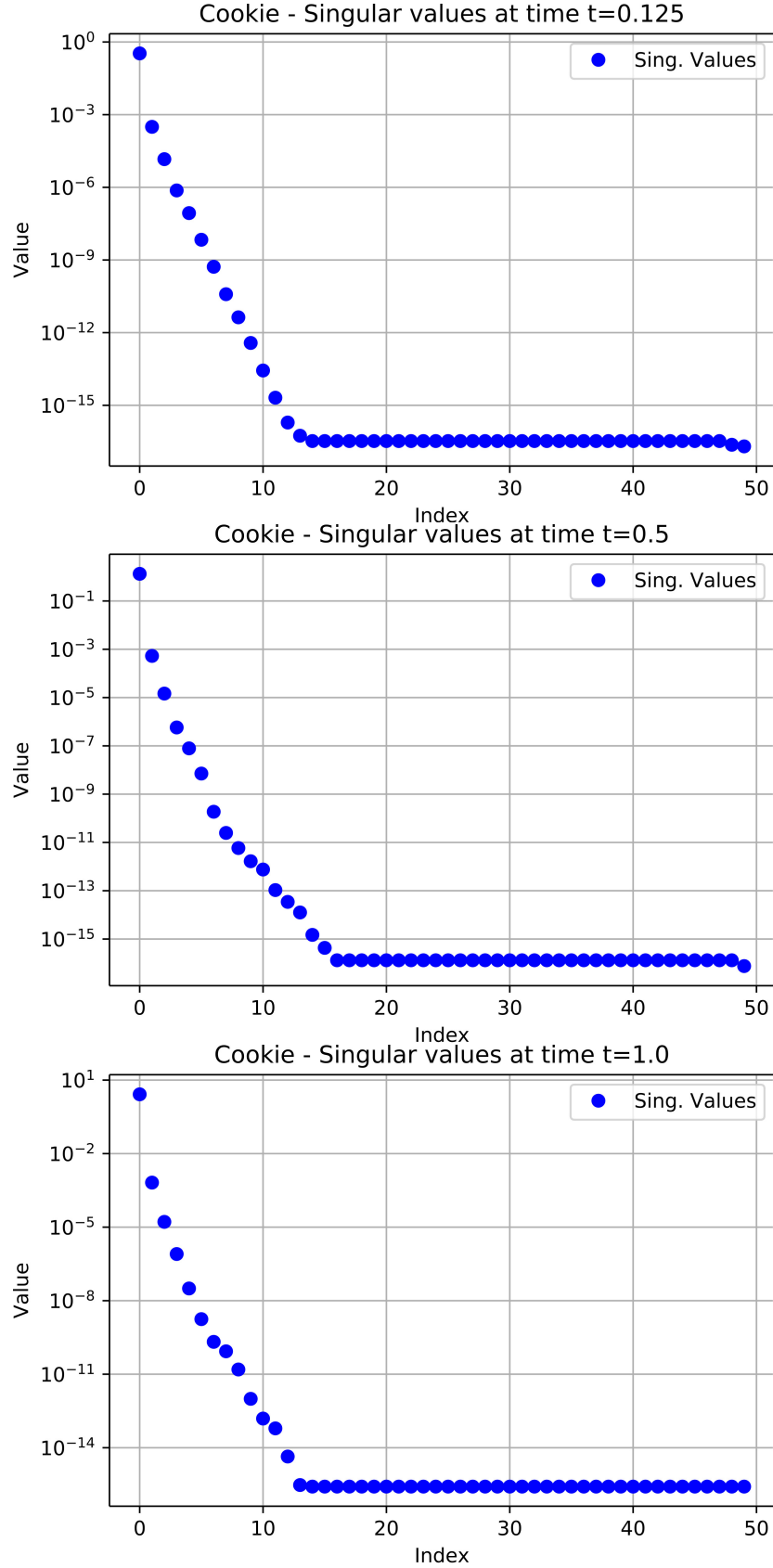


Figure 5.6: Singular values over time of the exact solution $Y(t)$ of the cookie problem with $p = 50$ different parameters.

The form of the problem is

$$\dot{X} = AX + BXC + D, \quad (5.10)$$

for general matrices A, B and C . Due to the two matrices B and C , this problem is not similar to the Lyapunov substep (5.2) and it cannot be solved by the Sylvester equation as we did for (5.3). To find its closed form solution, we first focus on the homogenous problem

$$\dot{X} = AX + BXC.$$

In vectorized form (stack columns under each-other), this equation can be written as

$$\dot{\mathbf{x}} = \text{vec}(\dot{X}) = (I \otimes A + C^T \otimes B)\text{vec}(X) =: \mathcal{L}\text{vec}(X) = \mathcal{L}\mathbf{x}. \quad (5.11)$$

Assuming that C is diagonalisable, by eigenvalue decomposition, we have

$$C = Q\Lambda Q^{-1}, \text{ or } C = Q\Lambda Q^T \text{ if } C \text{ is symmetric,}$$

so

$$\mathcal{L} = (Q \otimes I)(I \otimes A + \Lambda \otimes B)(Q^{-1} \otimes I) =: (Q \otimes I)\mathcal{A}(Q^{-1} \otimes I).$$

By definition of the Kronecker product, we have

$$\mathcal{A} = (I \otimes A + \Lambda \otimes B) = \begin{bmatrix} A + \lambda_1 B & & \\ & \ddots & \\ & & A + \lambda_n B \end{bmatrix}.$$

So we have the solution of the homogenous vectorized problem (5.11) as

$$\mathbf{x}(t) = \exp((t - t_0)\mathcal{L})\mathbf{x}_0 = (Q \otimes I)\exp((t - t_0)\mathcal{A})(Q^{-1} \otimes I)\mathbf{x}_0. \quad (5.12)$$

Now, by the variation of constant formula, we can find the solution of equation (5.10).

Theorem 5.1 (Variation of constant) *The solution of*

$$\begin{cases} \dot{\mathbf{y}} = A(t)\mathbf{y} + \mathbf{g}(t), \\ \mathbf{y}(t_0) = \mathbf{y}_0, \end{cases} \quad (5.13)$$

is given by

$$\mathbf{y}(t) = R(t, t_0)\mathbf{y}_0 + \int_{t_0}^t R(t, s)\mathbf{g}(s)ds,$$

where the solution of $\dot{\mathbf{y}} = A(t)\mathbf{y}$ is $\mathbf{y}(t) = R(t, t_0)\mathbf{c}$.

In our case, $A(t) = \mathcal{L}$ and $\mathbf{g}(t) = \text{vec}(D) =: \mathcal{D}$. We already found

$$\mathbf{x}(t) = \exp((t - t_0)\mathcal{L}) \mathbf{x}_0 =: R(t, t_0) \mathbf{x}_0.$$

Then, the solution is given by replacing $R(t, t_0)$ in formula (5.13), i.e.,

$$\mathbf{x}(t) = \exp((t - t_0)\mathcal{L}) \mathbf{x}_0 + \int_{t_0}^t \exp((t - t_0)\mathcal{L}) \mathcal{D} ds. \quad (5.14)$$

Compute the integral

$$\begin{aligned} \int_{t_0}^t \exp((t - t_0)\mathcal{L}) \mathcal{D} ds &= \int_{t_0}^t (Q \otimes I) \exp((t - t_0)\mathcal{A}) (Q^{-1} \otimes I) \mathcal{D} ds \\ &= (Q \otimes I) \exp(t\mathcal{A}) \int_{t_0}^t \exp(-s\mathcal{A}) (Q^{-1} \otimes I) \mathcal{D} ds \\ &= (Q \otimes I) (-\mathcal{A}^{-1} + \mathcal{A}^{-1} \exp((t - t_0)\mathcal{A})) (Q^{-1} \otimes I) \mathcal{D}. \end{aligned}$$

Finally, we insert this result and (5.12) into (5.14) to have the solution expressed with $\mathcal{A}$.

To conclude, we can compute efficiently the exact solution of the ODE

$$\dot{X} = AX + BXC + D,$$

by the following steps:

1. Compute the EVD

$$C = Q\Lambda Q^{-1}.$$

2. Define

$$\mathcal{A} = (I \otimes A + \Lambda \otimes B) = \begin{bmatrix} A + \lambda_1 B & & \\ & \ddots & \\ & & A + \lambda_n B \end{bmatrix}.$$

3. Compute efficiently the matvec product

$$\text{vec}(M_1) = \mathcal{A}^{-1} \text{vec}(DQ^{-T}).$$

4. Compute efficiently the exponential matvec product

$$\text{vec}(M_2) = \exp((t - t_0)\mathcal{A}) \text{vec}(M_1).$$

5. Compute efficiently the exponential matvec product

$$\text{vec}(M_3) = \exp((t - t_0)\mathcal{A})\text{vec}(X_0Q^{-T}).$$

6. Assemble the solution

$$X(t) = (M_3 + M_2 - M_1)Q^T.$$

Remark 5.3 (*Efficient computing*)

- *Inverse.* Since $\mathcal{A}$ is block-diagonal, we can efficiently compute the matvec product

$$\mathcal{A}^{-1}\tilde{\mathcal{D}} = \begin{bmatrix} \mathcal{A}_1^{-1}\tilde{\mathcal{D}}(:, 1) \\ \vdots \\ \mathcal{A}_n^{-1}\tilde{\mathcal{D}}(:, n) \end{bmatrix},$$

where we defined $\tilde{\mathcal{D}} := (Q^{-1} \otimes I)\mathcal{D} = \text{vec}(DQ^{-T})$. So we have n sparse linear systems to solve.

- *Matrix exponential.* Similarly, since $\mathcal{A}$ is block-diagonal, we can decompose

$$\exp((t - t_0)\mathcal{A})\mathcal{C} = \begin{bmatrix} \exp((t - t_0)\mathcal{A}_1)\mathcal{C}(:, 1) \\ \vdots \\ \exp((t - t_0)\mathcal{A}_n)\mathcal{C}(:, n) \end{bmatrix},$$

where $\mathcal{C}$ is a generic vector. Then use Remark 5.1 to compute efficiently the n exponential matvec products.

Since the substeps S and L are similar, the derivation also holds for these substeps. We used this method in our numerical experiments.

5.2.3 Cookie problem: Multirank Parareal

In this section, we apply Multirank Parareal to the Cookie problem described in Section 5.2.1.

The first integrator is the KSL scheme. In Figure 5.7, we compare the convergence rate for different coarse ranks r_1 . As we can see, the convergence rate is better when the coarse rank is higher.

The second integrator is the PRK3 method. In Figure 5.8, we observe the same convergence as in Figure 5.7, which indicates that the convergence does not depend on the solver. However, this strongly suggests

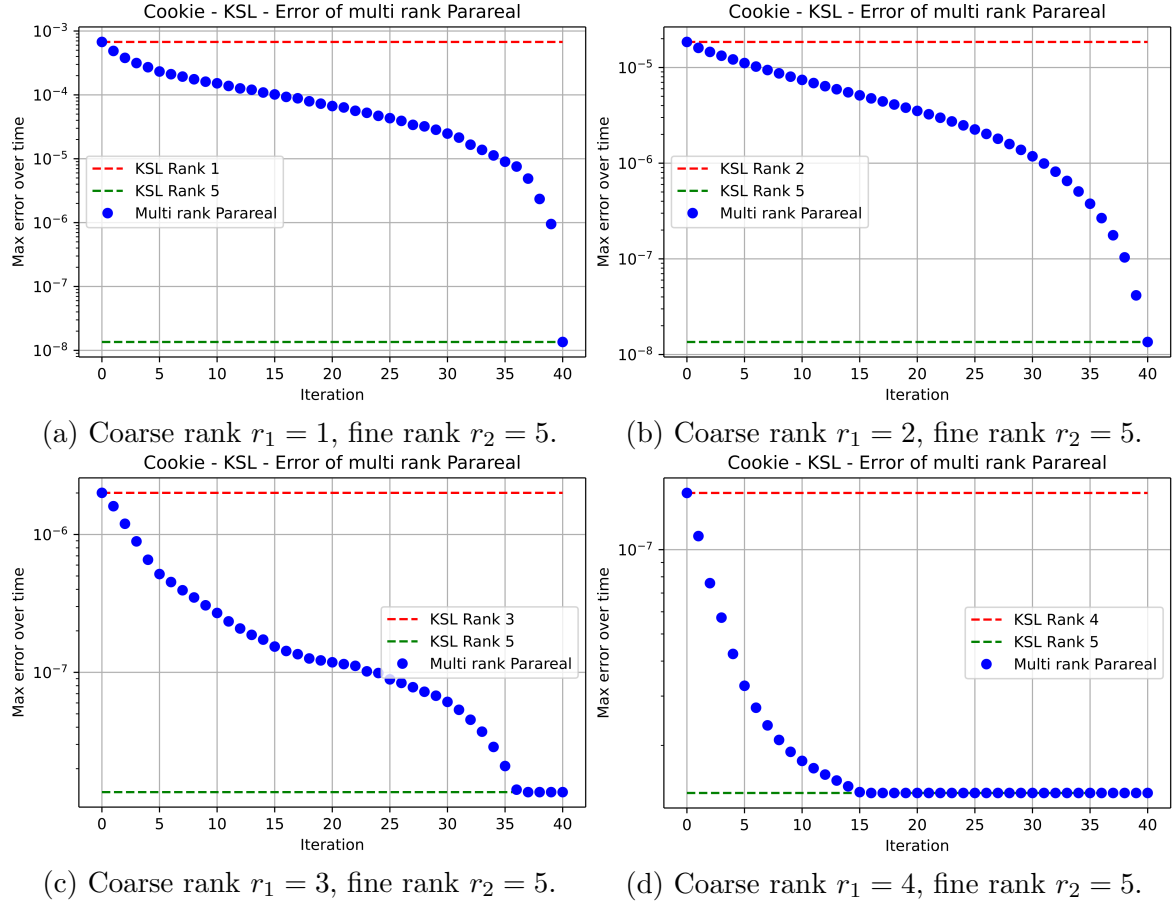


Figure 5.7: Multirank Parareal applied to the Cookie problem ($p = 50$) with the KSL scheme. When the coarse rank is larger, the convergence rate is better.

that the convergence of the Multirank Parareal algorithm depends on the problem itself. Our results suggest that Multirank Parareal does not work very well on parametric problems like the Cookie problem.

The third integrator is the robust projector splitting method (K-L:S scheme). In Figure 5.9, we can see that, even with this robust integrator, the convergence is not improved. It suggests again that the solvers are not the main point for the bad convergence of Multirank Parareal.

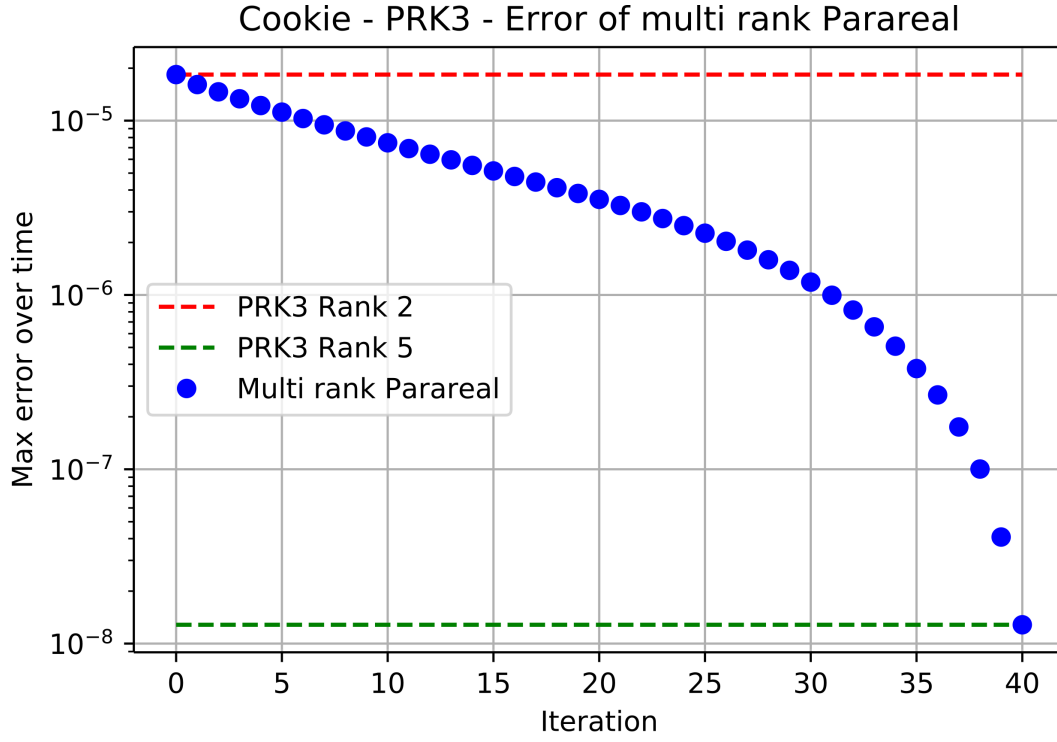


Figure 5.8: Multirank Parareal applied to the Cookie problem ($p = 50$) with the PRK3 method. The convergence is similar to the convergence with the KSL scheme.

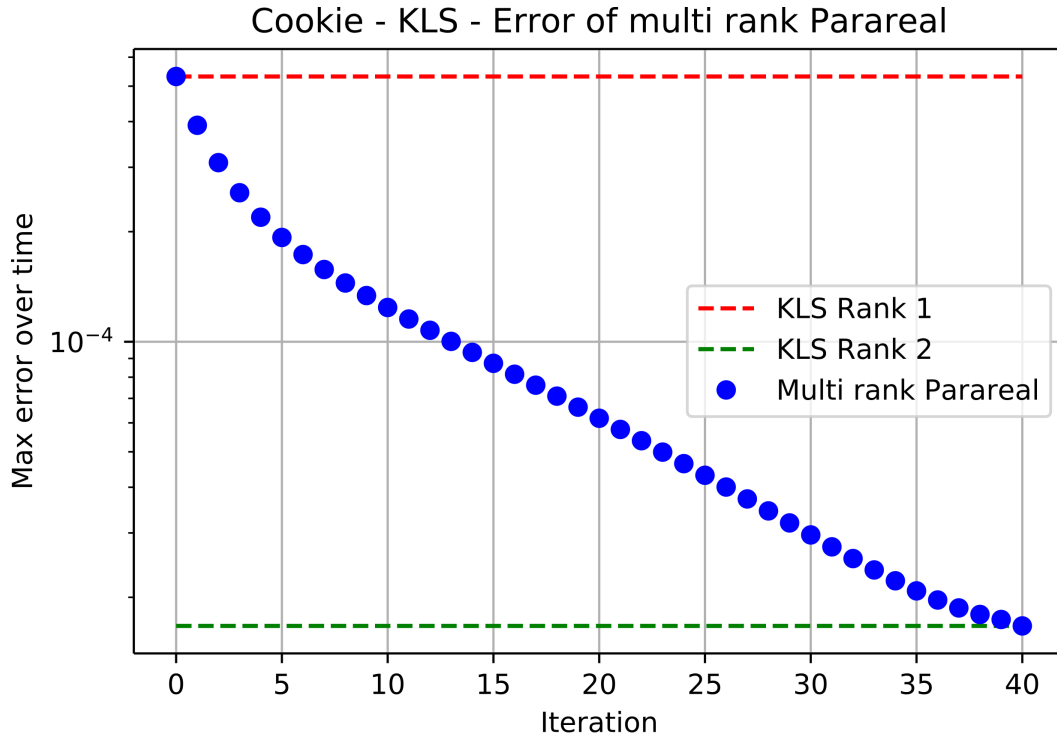


Figure 5.9: Multirank Parareal applied to the Cookie problem ($p = 30$) with the K-L:S scheme. The convergence is slow, there is no speed-up.

6 Conclusion

Inspired by the Parareal algorithm, we proposed a parallel time integrator for the DLRA problem. The key idea of this new algorithm is to use two different ranks: a coarse rank associated to the coarse solver and a fine rank associated to the fine solver. For this reason, we called this new algorithm *Multirank Parareal*. In fact, this algorithm solves the problem using two different levels of precision and, in this sense, is similar to a multigrid method where a coarse grid and a fine grid are used. Studying such algorithms is difficult because the coarse solver always brings an extra error in space. Despite this difficulty, we proposed an analysis where an assumption is necessary to get an interesting bound. This analysis led to a general theoretical bound that can be extended and studied for more specific problems.

We tested this new algorithm on two problems. The first problem was the heat problem, solved by the Lyapunov equation. In this case, the Multirank Parareal algorithm works well and we observed a speed-up between 2 and 6. It seems that the convergence rate depends on the choice of the coarse rank. A higher coarse rank seems to give a better convergence rate. This is consistent with the analysis, where the role of the coarse rank is important. In comparison, the choice of the solvers seems to have a very limited impact on the convergence rate. The second problem was a parametric problem for heat diffusion. In this case, the Multirank Parareal was less effective and we did not always obtain a speed-up.

Our numerical experiments suggest that the Multirank Parareal algorithm has similar properties to the original Parareal algorithm. In particular, this algorithm should work well on parabolic problems but not on hyperbolic problems.

The next step would be to find a sharp bound for a specific problem, like the heat problem. It seems also possible to study the Parareal algorithm in a more general context, using the Dahlquist's test equation. We also could study more precisely the assumptions made in the analysis in order to understand the role of the coarse rank in the convergence. Finally, we could find the optimal coarse rank in order to optimize the cost and the convergence rate.

References

- [1] A. H. AL-MOHY AND N. J. HIGHAM, *Computing the Action of the Matrix Exponential, with an Application to Exponential Integrators*, SIAM Journal on Scientific Computing, 33 (2011), pp. 488–511.
- [2] G. CERUTI AND C. LUBICH, *An unconventional robust integrator for dynamical low-rank approximation*.
- [3] C. ECKART AND G. YOUNG, *The approximation of one matrix by another of lower rank*, Psychometrika, 1 (1936), pp. 211–218.
- [4] M. J. GANDER, *Time Parallel Time Integration*, in Time Parallel Time Integration, University of Geneva, 2018.
- [5] M. J. GANDER, F. KWOK, AND H. ZHANG, *Multigrid interpretations of the parareal algorithm leading to an overlapping variant and MGRIT*, Computing and Visualization in Science, 19 (2018), pp. 59–74.
- [6] M. J. GANDER AND M. NEUMÜLLER, *Analysis of a New Space-Time Parallel Multigrid Algorithm for Parabolic Problems*, SIAM Journal on Scientific Computing, 38 (2016), pp. A2173–A2208.
- [7] N. HALKO, P.-G. MARTINSSON, AND J. A. TROPP, *Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions*, arXiv:0909.4061 [math], (2010). arXiv: 0909.4061.
- [8] E. KIERI AND B. VANDEREYCKEN, *Projection Methods for Dynamical Low-Rank Approximation of High-Dimensional Problems*, Computational Methods in Applied Mathematics, 19 (2019), pp. 73–92.
- [9] O. KOCH AND C. LUBICH, *Dynamical Low-Rank Approximation*, SIAM Journal on Matrix Analysis and Applications, 29 (2007), pp. 434–454.
- [10] T. G. KOLDA AND B. W. BADER, *Tensor Decompositions and Applications*, SIAM Review, 51 (2009), pp. 455–500.

- [11] R. KRAUSE, B. MÜLLER, AND G. STARKE, *An adaptive least-squares mixed finite element method for the Signorini problem: Adaptive Least-Squares For The Signorini Problem*, Numerical Methods for Partial Differential Equations, 33 (2017), pp. 276–289.
- [12] D. KRESSNER AND C. TOBLER, *Low-Rank Tensor Krylov Subspace Methods for Parametrized Linear Systems*, SIAM Journal on Matrix Analysis and Applications, 32 (2011), pp. 1288–1316.
- [13] A. S. LEWIS AND J. MALICK, *Alternating Projections on Manifolds*, Mathematics of Operations Research, 33 (2008), pp. 216–234.
- [14] J.-L. LIONS, Y. MADAY, AND G. TURINICI, *Résolution d’EDP par un schéma en temps pararéel*, Comptes Rendus de l’Académie des Sciences - Series I - Mathematics, 332 (2001), pp. 661–668.
- [15] T. LUNET, J. BODART, S. GRATTON, AND X. VASSEUR, *Time-parallel simulation of the decay of homogeneous turbulence using Parareal with spatial coarsening*, Computing and Visualization in Science, 19 (2018), pp. 31–44.
- [16] Y. NAKATSUKASA, *Fast and stable randomized low-rank matrix approximation*, arXiv:2009.11392 [cs, math], (2020). arXiv:2009.11392.
- [17] E. J. NYSTRÖM, *Über Die Praktische Auflösung von Integralgleichungen mit Anwendungen auf Randwertaufgaben*, Acta Mathematica, 54 (1930), pp. 185–204.
- [18] A. OSTERMANN, C. PIAZZOLA, AND H. WALACH, *Convergence of a Low-Rank Lie–Trotter Splitting for Stiff Matrix Differential Equations*, SIAM Journal on Numerical Analysis, 57 (2019), pp. 1947–1966.
- [19] R. B. SIDJE, *Expokit: a software package for computing matrix exponentials*, ACM Transactions on Mathematical Software, 24 (1998), pp. 130–156.
- [20] V. SIMONCINI, *Computational Methods for Linear Matrix Equations*, SIAM Review, 58 (2016), pp. 377–441.

- [21] A. USCHMAJEV AND B. VANDEREYCKEN, *Geometric Methods on Low-Rank Matrix and Tensor Manifolds*, in Handbook of Variational Methods for Nonlinear Geometric Data, P. Grohs, M. Holler, and A. Weinmann, eds., Springer International Publishing, Cham, 2020, pp. 261–313.